

Intelligenza Artificiale

Note Complete del Corso

Università di Torino – Magistrale Informatica
Prof. Gian Luca Pozzato – Prof. Roberto Micalizio

Indice

I	Programmazione Logica e Ragionamento (Prof. Gian Luca Pozzato)	4
1	Introduzione	4
2	Ricerca nello Spazio degli Stati	5
2.1	Formalizzazione del Problema di Ricerca	5
2.2	Strategie di Ricerca non Informata	6
3	Ricerca Euristica	7
3.1	Perché un Nodo Già Visitato Può Dover Essere Riaperto	7
3.2	Ammissibilità e Consistenza	7
4	Il Linguaggio Prolog	8
4.1	Programmazione Dichiarativa	8
4.2	Termini	8
4.3	Liste	8
5	Unificazione	10
5.1	L'Algoritmo di Unificazione	10
6	Risoluzione SLD e l'Interprete Prolog	12
6.1	Richiami: Risoluzione Classica	12
6.2	Unificazione nel Pattern Matching	12
6.3	Risoluzione SLD	12
6.4	Esempio: Leftmost vs. Rightmost	12
6.5	L'Interprete Prolog: Ciclo Risolutivo	13
7	Il Cut (!) in Prolog	15
7.1	Semantica del Cut	15
7.2	Un Esempio Completo: il Cambio di Semantica	15
7.3	Direzionalità del Cut	16

8	Answer Set Programming	17
8.1	ASP vs. Prolog	17
8.2	Sintassi	17
8.3	Semantica: Answer Set come Modello Minimale/Stabile	17
8.4	Risolvere Problemi con ASP: Generate-and-Test	18
II	Planning, Sistemi a Regole e Incertezza (Prof. Roberto Micalizio)	19
9	Fondamenti del Planning Automatico	19
9.1	Il Modello Concettuale: State Transition System	19
9.2	Le Assunzioni del Planning Classico	19
9.3	Il Problema di Planning Classico	20
9.4	Complessità	20
9.5	Proprietà di un Buon Algoritmo di Pianificazione	20
10	Rappresentazione a Variabili di Stato e Ricerca nello Spazio degli Stati	22
10.1	State-Variable Representation	22
10.2	Progression: Ricerca in Avanti	22
10.3	Regression: Ricerca all'Indietro	23
11	STRIPS	24
11.1	Stati e Operatori	24
11.2	L'Algoritmo STRIPS	24
11.3	Esempio Completo: Blocks World	25
11.4	Limiti del Linear Planning	25
12	PDDL	26
12.1	Esempio Completo: Gripper	26
12.2	Requirements	27
12.3	Tipi e Gerarchie	27
12.4	Effetti Condizionali e Quantificatori	28
12.5	Due Problemi di Modellazione Ricorrenti	28
12.6	Estensioni di PDDL	28
13	Pianificazione nello Spazio dei Piani	29
13.1	Formalizzazione del Piano Parziale	29
13.2	Flaws: Open Goal	29
13.3	Flaws: Threat	29
13.4	Algoritmo PSP	29
13.5	Esempio Completo: Scambio di Posizione tra Due Robot	30
13.6	Glossario	30
14	Grafi di Pianificazione e l'Algoritmo GraphPlan	31
14.1	Mutua Esclusione (Mutex)	31
14.2	GraphPlan come Euristica	31
14.3	L'Algoritmo GraphPlan	32
14.4	Esempio: "Have Cake and Eat It Too"	32
14.5	Terminazione	33
14.6	Proprietà	33

15 Euristiche per la Pianificazione	34
15.1 Euristiche per la Ricerca in Avanti	34
15.2 Euristiche per la Ricerca nello Spazio dei Piani	35
16 Hierarchical Task Networks (HTN)	36
17 Sistemi Esperti	37
17.1 Architettura	37
17.2 Forward vs. Backward Chaining	37
17.3 Domini Applicativi	37
18 CLIPS	39
18.1 Fatti	39
18.2 Regole	39
18.3 Un Esempio Completo: Caricamento ed Esecuzione	40
18.4 Pattern Matching Avanzato	41
18.5 Strategie di Conflict Resolution	41
18.6 Espressioni Antecedenti Complesse	42
18.7 Modellare un Mondo Dinamico	43
18.8 Input/Output	44
18.9 Programmazione Modulare	44
18.10 Ricerca con Backtracking in CLIPS	45
18.11 Incertezza nei Sistemi a Regole: i Certainty Factor	45
18.12 Caso di Studio: Sistema Esperto per la Scelta del Vino	46
19 Incertezza e Ragionamento Probabilistico	48
19.1 Perché la Logica Classica non Basta	48
19.2 La Probabilità come Soluzione	48
19.3 Notazione e Assiomi	48
19.4 Probabilità Condizionata	49
19.5 Regola di Bayes	49
19.6 Indipendenza e Indipendenza Condizionale	49
19.7 Reti Bayesiane: Sintassi e Semantica	50
19.8 Semantica Globale e Compattezza	50
19.9 Semantica Locale e Markov Blanket	51
19.10 Costruire una Rete Bayesiana	51
19.11 Inferenza per Enumerazione	51
19.12 Eliminazione delle Variabili	51
19.13 MPE e MAP	52

Parte I

Programmazione Logica e Ragionamento (Prof. Gian Luca Pozzato)

1 Introduzione

L'**intelligenza artificiale** è una disciplina dell'informatica che studia i fondamenti teorici, le metodologie e le tecniche che consentono di progettare sistemi hardware e software capaci di fornire prestazioni che, a un osservatore comune, sembrerebbero di pertinenza esclusiva dell'intelligenza umana (definizione di Marco Somalvico). In modo più operativo, l'IA si occupa di **problemi "intelligenti"**: problemi per cui non esiste, o non è noto, un algoritmo di risoluzione esplicito – si pensi alla differenza tra risolvere un cubo di Rubik seguendo una guida passo-passo (un algoritmo noto) e riconoscere un oggetto in un'immagine (nessun algoritmo esplicito, serve un sistema che "impari" o "ragioni").

Le principali sotto-aree di ricerca dell'IA sono la rappresentazione della conoscenza e il ragionamento, l'interpretazione e la sintesi del linguaggio naturale, l'apprendimento automatico, la pianificazione e la robotica; la disciplina attinge inoltre a filosofia, fisica e psicologia. Questa prima parte del corso si concentra sui **formalismi logici per la rappresentazione della conoscenza e il ragionamento**: meccanismi di ragionamento basati sul calcolo dei predicati del primo ordine, programmazione logica, ragionamento non monotono e Answer Set Programming, sperimentati concretamente in Prolog e in Clingo.

Vale la pena motivare fin da subito perché un corso del 2025/26 dedichi tempo a strumenti "classici" come Prolog e la logica, piuttosto che (solo) alle tecniche sub-simboliche oggi di moda. Un semplice esperimento chiarisce il punto: chiedendo a un moderno modello linguistico (LLM) di citare testualmente un'ottava della *Gerusalemme Liberata* di Tasso, il sistema produce con sicurezza tre citazioni diverse, tutte plausibili nello stile ma tutte inventate e diverse dal testo originale, anche dopo essere stato corretto ripetutamente dall'utente – un esempio di *allucinazione* sicura di sé, difficile da correggere dall'interno del sistema stesso. I metodi simbolici che vedremo in questa parte del corso (Prolog, risoluzione, ASP) hanno il pregio opposto: ogni risposta è tracciabile, verificabile e spiegabile passo per passo a partire da una base di conoscenza esplicita.

Un secondo esempio, complementare, mostra invece i limiti della logica classica (non probabilistica, non graduata) di fronte al **ragionamento di default**: "i lavoratori normalmente pagano l'IRPEF; gli studenti normalmente non la pagano; Gianni è uno studente lavoratore: Gianni paga l'IRPEF?" Le due regole di default sono in conflitto su Gianni, e senza un meccanismo che stabilisca una priorità tra regole (es. "più specifica vince") non c'è una risposta univoca. Questo è esattamente il tipo di ragionamento – **non monotono**, con regole di default che possono essere ritratte alla luce di nuova informazione – che la logica classica non gestisce nativamente, e che motiva gli strumenti più espressivi (in particolare l'Answer Set Programming) che vedremo più avanti.

2 Ricerca nello Spazio degli Stati

2.1 Formalizzazione del Problema di Ricerca

Un problema di ricerca è definito da: uno **stato iniziale**; un insieme di **azioni** (ciascuna fa passare da uno stato a un altro); una specifica dell'**obiettivo** (goal); il **costo** di ogni azione. Lo **spazio degli stati** è definito *implicitamente* da stato iniziale e azioni: è l'insieme di tutti gli stati raggiungibili a partire da quello iniziale. Un **cammino** è una sequenza di stati collegati da una sequenza di azioni, e il suo **costo** è la somma dei costi delle azioni che lo compongono. Una **soluzione** è un cammino dallo stato iniziale a uno stato goal; una **soluzione ottima** è una soluzione di costo minimo tra tutte le soluzioni.

In Prolog, gli stati si rappresentano come termini (dipendenti dal problema: ad esempio `on(a,b)` nel mondo dei blocchi, oppure una lista ordinata per il puzzle dell'8). Le azioni si specificano tramite due predicati standard: `applicabile(Az,S)` (l'azione `Az` è eseguibile nello stato `S`) e `trasforma(Az,S,NuovoS)` (se `Az` è applicabile a `S`, `NuovoS` è il risultato). Un'istanza di problema è definita da `iniziale(S)` e `finale(S)`.

Un caso di studio ricorrente è la ricerca di un cammino in un **labirinto** rappresentato come griglia con ostacoli: la posizione è `pos(Riga,Colonna)`, gli ostacoli sono `occupata(pos(Riga,Colonna))`, e le azioni sono i quattro spostamenti cardinali. Un'azione è applicabile quando non porta l'agente fuori dalla griglia né su una cella occupata:

```
num_col(10).
num_righe(10).
occupata(pos(2,5)). occupata(pos(7,4)).
occupata(pos(3,5)). occupata(pos(5,7)).
% ... altri ostacoli ...
iniziale(pos(4,2)).
finale(pos(7,9)).

applicabile(nord,pos(R,C)) :-
    R>1, R1 is R-1, \+ occupata(pos(R1,C)).
applicabile(sud,pos(R,C)) :-
    num_righe(NR), R<NR, R1 is R+1, \+ occupata(pos(R1,C)).
applicabile(ovest,pos(R,C)) :-
    C>1, C1 is C-1, \+ occupata(pos(R,C1)).
applicabile(est,pos(R,C)) :-
    num_col(NC), C<NC, C1 is C+1, \+ occupata(pos(R,C1)).

trasforma(est,pos(R,C),pos(R,C1)) :- C1 is C+1.
trasforma(ovest,pos(R,C),pos(R,C1)) :- C1 is C-1.
trasforma(sud,pos(R,C),pos(R1,C)) :- R1 is R+1.
trasforma(nord,pos(R,C),pos(R1,C)) :- R1 is R-1.
```

Il codice sopra usa già due costrutti Prolog che formalizzeremo solo più avanti (Sezione 4), ma che è utile capire subito per leggere il codice: `is/2` valuta *aritmeticamente* l'espressione a destra e unifica il risultato con il termine a sinistra (es. `R1 is R-1` calcola `R-1` e lega `R1` al risultato numerico – a differenza di `=`, che farebbe solo unificazione simbolica senza fare alcun calcolo); `\+ G` è la **negazione per fallimento**: ha successo se e solo se il goal `G` *fallisce* (non è dimostrabile dal programma corrente), quindi `\+ occupata(pos(R1,C))` ha successo esattamente quando la cella non è nella lista degli ostacoli.

2.2 Strategie di Ricerca non Informata

Il vantaggio di Prolog è che la **ricerca in profondità** (DFS) è, in un certo senso, "gratuita": l'interprete realizza di per sé una ricerca in profondità con backtracking sfruttando il proprio nondeterminismo nativo. Una versione minimale, che non controlla i cicli, è:

```
ric_prof(S,[]) :- finale(S), !.
ric_prof(S,[Az|Resto]) :-
    applicabile(Az,S),
    trasforma(Az,S,Nuovo_S),
    ric_prof(Nuovo_S,Resto).
```

Il secondo argomento accumula la sequenza di azioni (il piano soluzione). Il taglio ! nella prima clausola non è decorativo: senza di esso, una volta trovato un successo con `finale(S)`, un successivo backtracking (ad esempio richiesto da chi ha invocato `ric_prof`) proverebbe comunque anche la seconda clausola a partire da uno stato che è già finale, producendo rami di ricerca inutili o ulteriori soluzioni ridondanti; il cut congela la scelta "se lo stato è finale, il piano è [] e basta" impedendo di esplorare oltre. Per evitare cicli infiniti (rivisitare uno stato già incontrato) si mantiene esplicitamente una lista `Visitati` (l'analogo di una *closed list*):

```
ric_prof_cc(S,_,[]) :- finale(S),!.
ric_prof_cc(S,Visitati,[Az|Resto]) :-
    applicabile(Az,S),
    trasforma(Az,S,Nuovo_S),
    \+ member(Nuovo_S,Visitati),
    ric_prof_cc(Nuovo_S,[Nuovo_S|Visitati],Resto).
```

Qui `member(Elemento,Lista)` è un predicato built-in che ha successo se `Elemento` compare in `Lista` (lo si vedrà formalmente in Sezione 4): la condizione `\+ member(Nuovo_S,Visitati)` scarta quindi ogni successore già visitato, prima ancora di proseguire la ricorsione.

Oltre alla DFS pura, si distinguono altre strategie non informate: la **ricerca a profondità limitata** (come la DFS, ma con un limite massimo ℓ alla profondità oltre il quale i nodi non vengono espansi) è **non completa**; l'**iterative deepening** (o approfondimento iterativo) ripete la ricerca a profondità limitata incrementando ℓ ad ogni iterazione, ed è **ottima nel caso di azioni a costo unitario**; la **ricerca in ampiezza** (BFS) usa una **coda FIFO** (il nodo in testa viene espanso, i successori vanno in fondo alla coda) – va notato che, a differenza della DFS, la BFS *non* è "gratuita" sfruttando il solo backtracking di Prolog: richiede di generare esplicitamente *tutti* i successori di un nodo (tipicamente con `findall`) prima di espandere il nodo successivo in coda; inoltre è ottima solo alla stessa condizione di costo uniforme/unitario dell'iterative deepening, non in generale. Infine la **ricerca in ampiezza su grafi** aggiunge una **lista chiusa** dei nodi già espansi, analoga al meccanismo `Visitati` visto per la DFS.

Per riferimento, valgono le proprietà standard di completezza/ottimalità e complessità (con b fattore di ramificazione, d profondità della soluzione, m profondità massima dell'albero di ricerca, ℓ limite di profondità imposto):

Strategia	Completa	Ottima	Tempo	Spazio
BFS (costo unitario)	sì	sì	$O(b^d)$	$O(b^d)$
DFS	no (spazi infiniti)	no	$O(b^m)$	$O(bm)$
Profondità limitata	no	no	$O(b^\ell)$	$O(b\ell)$
Iterative deepening	sì	sì (costo unit.)	$O(b^d)$	$O(bd)$

Una **funzione euristica** $h(n)$ stima il costo del cammino più conveniente dal nodo n a uno stato finale; $g(n)$ è il costo effettivo del cammino trovato dal nodo iniziale a n ; la **funzione di valutazione** è $f(n) = g(n) + h(n)$. L'algoritmo **A*** usa $f(n)$ per scegliere quale nodo espandere (il minimo tra quelli in frontiera); **IDA*** (Iterative Deepening A*) applica la stessa idea dell'iterative deepening ma con una soglia stimata anziché un limite di profondità intero: al primo passo la soglia è $h(s_i)$ (stato iniziale), e ad ogni iterazione la nuova soglia è il minimo $f(n) = g(n) + h(n)$ tra i nodi che avevano superato la soglia precedente.

3.1 Perché un Nodo Già Visitato Può Dover Essere Riaperto

Un nodo può essere raggiunto da più cammini diversi, con costi g diversi. Se l'euristica usata garantisce solo l'ammissibilità (non la consistenza, vedi sotto), un nodo già espanso con un certo f può in seguito rivelarsi raggiungibile con un f migliore attraverso un cammino alternativo: va allora **riaperto** (rimesso in frontiera), e l'aggiornamento si propaga anche ai suoi discendenti già generati. Concretamente: un nodo radice con $h = 7$ genera due figli, uno con $g + \text{arco} + h = 0 + 4 + 2 = 6$ (il migliore secondo f) e uno con $0 + 1 + 6 = 7$; espandendo il primo si arriva a un discendente con $f = 4 + 2 + 1 = 7$. Se successivamente si scopre che lo stesso discendente è raggiungibile anche passando dal secondo figlio con g inferiore (ad esempio $f = 1 + 2 + 2 = 5$, contro il precedente $f = 6$ del suo genitore ricalcolato), il nodo genitore va aggiornato da $f = 6$ a $f = 5$, e il discendente da $f = 7$ a $f = 6$: entrambi vanno *riaperti* nonostante fossero già stati chiusi.

3.2 Ammissibilità e Consistenza

Sia $d(s)$ la distanza minima *effettiva* (non nota, altrimenti la seguiremmo direttamente) tra lo stato s e il goal:

$$h(s) \text{ è ammissibile} \iff 0 \leq h(s) \leq d(s)$$

e vale il teorema (senza dimostrazione sulle slide del corso): **A*** è ottimale se h è ammissibile.

$h(s)$ è **consistente** (o **monotona**) se, per ogni coppia s, s' con s' successore di s ,

$$h(s) \leq c(s, s') + h(s')$$

dove $c(s, s')$ è il costo per passare da s a s' . Se h è consistente allora è anche ammissibile, e **A*** è ottimale se h è consistente – con il vantaggio pratico che, in questo caso, **non è mai necessario riaprire** uno stato già visitato (la consistenza garantisce che il primo cammino trovato verso un nodo sia già quello di costo minimo).

Esempio – distanza di Manhattan in un labirinto. Se le azioni hanno tutte lo stesso costo e non sono ammessi movimenti diagonali, la distanza di Manhattan è monotona (consistente). Se invece sono ammesse anche mosse diagonali, non lo è più: si consideri un nodo con euristica $h = 6$; una mossa diagonale verso il goal può ridurre la distanza di Manhattan di 2 in una sola mossa di costo 1, portando a un figlio con $h = 4$. La condizione di consistenza richiederebbe $h(s) \leq c(s, s') + h(s')$, cioè $6 \leq 1 + 4 = 5$, che è **falsa**: la distanza di Manhattan con mosse diagonali resta ammissibile ma non è più consistente, e può quindi richiedere la riapertura di nodi già visitati.

4 Il Linguaggio Prolog

4.1 Programmazione Dichiarativa

Prolog (*PRO*gramming in *LOG*ic) è un **linguaggio dichiarativo**: non si descrive *cosa fare* per risolvere un problema (come nel paradigma imperativo/procedurale, che specifica una sequenza di passi), ma si descrive la situazione reale tramite **fatti** e **regole**, chiedendo poi all'interprete di verificare se un **goal** segue logicamente, secondo la logica classica.

Un **fatto** rappresenta un'informazione del dominio, ad esempio `piuCalorico(wurstel,banana)`. (il wurstel è più calorico della banana). Una **regola** rappresenta un'inferenza possibile, nella forma di una **clausola di Horn**:

```
testa :- corpo1, corpo2, ..., corpoN.
```

ad esempio `felino(X) :- gatto(X)`. (i gatti sono felini). L'interprete analizza fatti e regole nell'ordine in cui compaiono nel programma, usando il **pattern matching** (unificazione) per confrontare un goal con le teste di fatti/regole; nel caso di una regola, il matching genera **sotto-goal** dal corpo. Fatti e regole insieme sono **clausole**; un **programma Prolog** è un insieme finito di clausole, che viene interrogato con un goal.

4.2 Termini

Un programma Prolog è costruito da: **atomi** (costanti, come `verza`, `a`, o numeri); **variabili** (identificate dall'iniziale maiuscola); **termini composti**, ottenuti applicando un **funtore** ad altri termini, ad esempio `sopra(penna,tavolo)` o `nato(giovanni,data(torino,'12/07/1995'))` (si noti che una data va quotata come atomo se contiene caratteri come `/`, altrimenti Prolog la interpreta come un'espressione aritmetica).

4.3 Liste

Le **liste** sono la struttura dati principale di Prolog: `[1, ciao, 2]` è una lista di tre elementi; `[]` è la lista vuota; gli elementi possono a loro volta essere liste (liste annidate), es. `[23, [], ciao, [21, saluti], 11]`. Ogni lista non vuota è caratterizzata da una **testa** (il primo elemento) e una **coda** (il resto), rappresentata con la notazione `[Head|Tail]`. Questa scomposizione è alla base della programmazione ricorsiva su liste:

```
?- [1,2,3,4,5] = [Head|Tail].
Head = 1
Tail = [2,3,4,5]
```

la query unifica la lista concreta con il pattern `[Head|Tail]`: l'interprete lega **Head** al primo elemento e **Tail** al resto, e stampa i binding trovati. Alcuni predicati built-in fondamentali: `length(Lista,N)` (ha successo se `List`a contiene `N` elementi) e `member(Elemento,List)`a (ha successo se `List`a contiene `Elemento`). Come ogni altra operazione su liste, anche il calcolo di una somma si scrive per ricorsione su testa/coda:

```
somma([],0).
somma([Head|Tail],S) :- somma(Tail,S1), S is S1+Head.
```

(caso base: lista vuota, somma 0; passo ricorsivo: la somma è la testa più la somma ricorsiva della coda.) Il predicato `is/2` è l'operatore di **valutazione aritmetica**: `Risultato is Espressione` calcola il valore numerico di `Espressione` (che deve essere già completamente istanziata, cioè senza variabili libere) e lo unifica con `Risultato`. È diverso dall'unificazione pura `=`: la query `X = 2+3` lega semplicemente `X` al termine simbolico `2+3` (senza calcolarlo), mentre `X is 2+3` lega

X al numero 5. Nell'esempio di `somma`, invocando `somma([1,2,3],S)` l'interprete scende ricorsivamente fino al caso base (`somma([],0)`, che lega `S1=0` al livello più interno), e poi, risalendo, ogni chiamata calcola con `is` la propria somma parziale ($0+3=3$, poi $3+2=5$, infine $5+1=6$), fino a legare l'argomento `S` della chiamata originale a 6. Predicati simili, costruiti con lo stesso schema testa/coda, permettono di rimuovere un elemento (`select`), invertire una lista o concatenarne due (`append`).

L'**unificazione** è il meccanismo che permette a Prolog di confrontare due termini e trovare, se esiste, un modo di renderli identici. Una **sostituzione** è una funzione da variabili a termini, rappresentata come insieme finito di coppie $\sigma = \{x_1/t_1, \dots, x_n/t_n\}$ con le x_i variabili distinte. L'**applicazione** di σ a un termine t (notazione $t\sigma$) sostituisce *simultaneamente* ogni occorrenza di x_i con t_i : ad esempio, con $t = f(x_1, g(x_2), a)$ e $\sigma = \{x_1/h(x_2), x_2/b\}$, si ottiene $t\sigma = f(h(x_2), g(b), a)$ (la x_2 dentro $h(x_2)$ non viene ri-sostituita nello stesso passo, essendo l'applicazione simultanea).

Due termini t_1, t_2 sono **unificabili** se esiste una sostituzione σ (un **unificatore**) tale che $t_1\sigma = t_2\sigma$. Se due termini sono unificabili, esiste sempre un **unificatore più generale** (*most general unifier*, **mg**u), unico a meno di ridenominazione delle variabili: una sostituzione θ è più generale di σ se esiste λ tale che $\sigma = \theta\lambda$ (composizione, cioè applicazione in sequenza).

5.1 L'Algoritmo di Unificazione

Un problema di unificazione si formula come un insieme di equazioni $\{t'_j = t''_j\}$; la soluzione, se esiste, è l'mgu che le rende tutte vere. Si applicano ripetutamente due trasformazioni fino a raggiungere una **forma risolta** (equazioni tutte del tipo $x_j = t_j$, con ogni x_j che compare solo lì):

- **Term reduction**: data $f(t'_1, \dots, t'_n) = f(t''_1, \dots, t''_n)$ (stesso funtore), la si sostituisce con $\{t'_1 = t''_1, \dots, t'_n = t''_n\}$.
- **Variable elimination**: data $x = t$ (x variabile), si applica $\{x/t\}$ a *tutte le altre* equazioni (l'equazione $x = t$ resta).

Regole complete dell'algoritmo (non deterministico nella scelta dell'equazione su cui operare, termina con successo o fallimento):

- se un'equazione ha forma $t = x$ con t non-variabile e x variabile, la si riscrive $x = t$;
- se un'equazione ha forma $x = x$, la si cancella;
- se un'equazione ha forma $t' = t''$ con entrambi non-variabili: fallimento se i funtori principali differiscono, altrimenti term reduction;
- se un'equazione ha forma $x = t$ con x altrove nel sistema e $t \neq x$: fallimento se x occorre in t (**occur-check**), altrimenti variable elimination.

Esempio completo. Insieme di partenza $\{g(x_2) = x_1, f(x_1, h(x_1), x_2) = f(g(x_3), x_4, x_3)\}$:

$$\begin{array}{lcl}
 & & \{g(x_2) = x_1, f(x_1, h(x_1), x_2) = f(g(x_3), x_4, x_3)\} \\
 \xrightarrow{\text{term reduction}} & & \{g(x_2) = x_1, x_1 = g(x_3), h(x_1) = x_4, x_2 = x_3\} \\
 \xrightarrow{\text{var. elim. su } x_1=g(x_3)} & & \{g(x_2) = g(x_3), x_1 = g(x_3), h(g(x_3)) = x_4, x_2 = x_3\} \\
 \xrightarrow{\text{term reduction su } g(x_2)=g(x_3)} & & \{x_2 = x_3, x_1 = g(x_3), x_4 = h(g(x_3)), x_2 = x_3\} \\
 \xrightarrow{\text{var. elim. su } x_2=x_3} & & \{x_2 = x_3, x_1 = g(x_3), x_4 = h(g(x_3))\}
 \end{array}$$

Forma risolta raggiunta, con mgu

$$\theta = \{x_1/g(x_3), x_2/x_3, x_4/h(g(x_3))\}.$$

Per ragioni di efficienza, l'unificazione *standard di Prolog* non esegue l'*occur-check*: una variabile X unifica con $f(X)$, producendo potenzialmente termini circolari/infiniti. Questa è, esplicitamente, una strategia *non corretta* rispetto alla nozione logica di unificazione, accettata per ragioni prestazionali.

6.1 Richiami: Risoluzione Classica

Una formula F è **conseguenza logica** di una base di conoscenza K (notazione $K \models F$) se F è vera in tutti i modelli di K ; verificarlo con le tavole di verità è laborioso già con poche formule, da cui la necessità della **risoluzione**. La risoluzione si applica a formule in **forma di clausole** (disgiunzioni di letterali): date $C_1 = A_1 \vee \dots \vee A_n$ e $C_2 = B_1 \vee \dots \vee B_m$, se esistono A_i, B_j con $A_i = \neg B_j$, si deriva il **risolvente** (la disgiunzione di tutti i letterali di C_1, C_2 tranne A_i, B_j), conseguenza logica di $C_1 \wedge C_2$. Una **dimostrazione per refutazione** mostra che $K \wedge \neg F$ è inconsistente, risolvendo iterativamente coppie di clausole fino a ottenere la **clausola vuota** $\square$. In presenza di variabili, i letterali risolti vanno resi unificabili tramite un mgu σ , e il risolvente è $[\dots]\sigma$.

6.2 Unificazione nel Pattern Matching

Per l'efficienza dell'interprete Prolog è utile classificare tutti i casi di unificazione tra due termini c_1 (righe) e c_2 (colonne):

	costante c_2	variabile x_2	composto s_2
costante c_1	unificano sse $c_1=c_2$	unificano, x_2/c_1	non unificano
variabile x_1	unificano, x_1/c_2	unificano, x_1/x_2	unificano, x_1/s_2
composto s_1	non unificano	unificano, x_2/s_1	unificano sse stesso funtore e argomenti unificano

6.3 Risoluzione SLD

SLD = Linear resolution with Selection function for Definite clauses. Si applica quando K contiene solo **clausole definite** (clausole di Horn con al più un letterale non negato). Nella strategia *linear input*, ad ogni passo di risoluzione una **variante** (clausola con variabili rinominate) è sempre scelta dalla K di partenza, mentre l'altra è sempre il risolvente del passo precedente (al primo passo, la negazione del goal). **SLD non è completa in generale**, ma è **completa per clausole di Horn**.

Una **derivazione SLD** per un goal G_0 da K è una sequenza di goal $G_0, G_1, \dots, G_n$, una sequenza di varianti di clausole $C_1, \dots, C_n$ e una sequenza di mgu $\sigma_1, \dots, \sigma_n$, tali che G_{i+1} è derivato da G_i e C_{i+1} tramite σ_{i+1} . Una derivazione può avere **successo** ($G_n = \square$), **fallimento finito** (nessun risolvente possibile, $G_n \neq \square$), o **fallimento infinito** (non termina mai).

Ci sono **due forme distinte di nondeterminismo**: (1) la **regola di calcolo** (o *selection rule*), che seleziona quale atomo del goal risolvere ad ogni passo – non influenza correttezza e completezza; (2) la scelta di *quale clausola* di K usare tra quelle unificabili con l'atomo selezionato. Data una regola di calcolo, si rappresentano tutte le derivazioni possibili con un **albero SLD**: la radice è G_0 ; ogni nodo $\leftarrow A_1, \dots, A_m, \dots, A_k$ (con A_m atomo selezionato) ha un figlio per ogni clausola $A \leftarrow B_1, \dots, B_k$ di K tale che A e A_m unificano con mgu σ , etichettato $\leftarrow [A_1, \dots, A_{m-1}, B_1, \dots, B_k, A_{m+1}, \dots, A_k]\sigma$.

6.4 Esempio: Leftmost vs. Rightmost

Si considerino le clausole (con la corrispondente sintassi Prolog):

#	Clausola	Prolog
C1	$\neg PosaFacile(x) \vee \neg Economico(x) \vee Acquisto(x)$	<code>acquisto(X):-posaFacile(X),economico(X).</code>
C2	$\neg InOfferta(x) \vee Acquisto(x)$	<code>acquisto(X):-inOfferta(X).</code>
C3	$\neg InMagazzino(x) \vee Disponibile(x)$	<code>disponibile(X):-inMagazzino(X).</code>
C4	$InOfferta(mosaico)$	<code>inOfferta(mosaico).</code>
C5	$Disponibile(laminato)$	<code>disponibile(laminato).</code>
C6	$PosaFacile(laminato)$	<code>posaFacile(laminato).</code>
C7	$Economico(laminato)$	<code>economico(laminato).</code>
GOAL	$\neg Disponibile(x) \vee \neg Acquisto(x)$	<code>disponibile(X), acquisto(X).</code>

Con regola di calcolo **leftmost** (sempre l'atomo più a sinistra), dal goal si seleziona *Disponibile*: il ramo C3 fallisce subito (nessun fatto *InMagazzino*); il ramo C5 lega $x=laminato$ e passa a $\neg Acquisto(laminato)$, dove il ramo C1 (via C6, C7) porta a $\square$ in tre passi, mentre il ramo C2 fallisce (nessuna offerta su *laminato*). Con regola **rightmost** (atomo più a destra) l'ordine di esplorazione è completamente diverso: dal goal si seleziona per prima *Acquisto*, che si scinde nei rami C1 e C2. Il ramo C2 richiede $\neg InOfferta(x)$: unifica con C4 ($x=mosaico$), ma a quel punto resta da provare $\neg Disponibile(mosaico)$, che fallisce (nessun fatto *InMagazzino(mosaico)* né *Disponibile(mosaico)*) – l'intero ramo C2 fallisce. Il ramo C1 richiede invece $\neg PosaFacile(x) \vee \neg Economico(x)$: selezionando ancora da destra si prova prima C7 (*Economico(laminato)*, quindi $x=laminato$), poi C6 (*PosaFacile(laminato)*, verificato), e infine resta da provare $\neg Disponibile(laminato)$, soddisfatto direttamente da C5 – il ramo C1 porta a $\square$. Il risultato è quindi lo stesso della regola leftmost (un solo ramo di successo, via C1/C5/C6/C7), ma raggiunto attraversando l'albero SLD in un ordine completamente diverso: conferma pratica che la regola di calcolo non altera correttezza e completezza.

Ciò che rende Prolog **deterministico** è la combinazione di tre scelte fisse: regola di calcolo **leftmost**, clausole nell'**ordine in cui sono scritte** nel programma, ricerca **in profondità con backtracking**. Questo rende Prolog **non completo** in pratica: se un ramo infinito precede (a sinistra) un ramo di successo, l'interprete vi resta intrappolato e non trova mai la soluzione.

6.5 L'Interprete Prolog: Ciclo Risolutivo

L'interprete Prolog esegue un **backward chaining in profondità**, in forma non deterministica:

```

interprete(G)
if G vuoto
  then successo
  else sia G = G1, G2, ..., Gn
    sia R l'insieme delle clausole applicabili a G1
    if R vuoto
      then fallimento
      else scegliere una clausola A :- B1, ..., Bm in R
        sia sigma il MGU di G1 e A
        interprete(B1*sigma, ..., Bm*sigma, G2*sigma, ..., Gn*sigma)

```

Una regola $A \leftarrow B_1, \dots, B_m$ è *applicabile* a G_1 se le sue variabili vengono rinominate (standardizzazione apart) e A unifica con G_1 . La computazione ha successo se *esiste* una computazione che termina con successo (da qui il nondeterminismo); l'interprete Prolog reale, come visto, fissa deterministicamente quale scelta esplorare per prima.

Applicando questo ciclo, passo dopo passo, al goal `disponibile(X), acquisto(X)` dell'esempio precedente: (1) $G_1 = \text{disponibile}(X)$, l'unica clausola applicabile in ordine di programma con testa unificabile è C3 (`disponibile(X):-inMagazzino(X)`), che sostituisce il goal con `inMagazzino(X), acquisto(X)`; (2) $G_1 = \text{inMagazzino}(X)$ non ha alcuna clausola applicabile (R vuoto): **fallimento**, e l'interprete fa backtracking sulla scelta (1); (3) tornando al passo (1), la clausola successiva con testa unificabile è C5 (`disponibile(laminato).`, un fatto), che lega

$X=laminato$ e riduce il goal a `acquisto(laminato)`; (4) $G_1=acquisto(laminato)$, si prova prima C1 (`acquisto(X):-posaFacile(X),economico(X).`), che riduce il goal a `posaFacile(laminato), economico(laminato)`; (5) entrambi sono fatti (C6, C7): il goal diventa vuoto, **successo**, con risposta $X=laminato$. Si noti che l'interprete non ha mai dovuto considerare C2 né la clausola C1 fallita in altri contesti: il backtracking è scattato una sola volta, al passo (2), esattamente come previsto dal ciclo `interprete(G)`.

7 Il Cut (!) in Prolog

Il **cut** (!) è un predicato **extra-logico**: sempre vero se eseguito, ma con l'effetto collaterale di **bloccare il backtracking**, cioè di rendere definitive le scelte fatte. È utile modellare l'esecuzione Prolog con due stack: uno di **esecuzione** (i record di attivazione dei predicati correnti) e uno di **backtracking** (i punti di scelta, o *choice point*, ancora aperti) – in pratica un unico stack con alternanza di questi due tipi di elementi.

7.1 Semantica del Cut

Data una regola $p : -q_1, \dots, q_i, !, q_{i+1}, \dots, q_n$, quando l'esecuzione raggiunge il cut: la sua valutazione ha sempre successo; il cut viene ignorato in fase di backtracking; **tutte le scelte fatte nella valutazione di $q_1, \dots, q_i$ e nella scelta della clausola per p vengono rese definitive** (i relativi choice point sono rimossi dallo stack di backtracking). Le alternative per i goal *successivi* al cut ($q_{i+1}, \dots, q_n$) restano invece disponibili per il normale backtracking. Conseguenza importante: se la valutazione di $q_{i+1}, \dots, q_n$ fallisce, **fallisce l'intera valutazione di p** , anche se esisterebbero altre soluzioni logicamente valide – perché quelle alternative sono state irrevocabilmente tagliate dal cut. In breve: **il cut taglia rami dell'albero SLD**, e per questo può causare una **perdita di completezza** (rami che porterebbero a successo diventano irraggiungibili) oltre a una **perdita di dichiaratività** (il comportamento del programma dipende ora dall'ordine di scrittura e dal cut, non solo dalla logica).

7.2 Un Esempio Completo: il Cambio di Semantica

Si considerino le clausole:

```
g :- a.  
g :- s.  
a :- p, !, b.  
a :- r.  
p :- q.  
p :- r.  
r.
```

Query :- g. Senza cut, il backtracking esplorerebbe: a via p (che fallisce su q ma ha successo via r), poi b (che fallisce); a questo punto, *senza* cut, Prolog ritenterebbe la seconda clausola di a (a:-r.), che ha successo tramite il fatto r., rendendo vero a e quindi g (tramite la prima clausola di g). **Con** il cut nella clausola a:-p,!,b., invece, una volta che p ha successo (tramite r, dopo che q è fallito) e si raggiunge !, vengono rimossi *sia* gli eventuali choice point residui per p (se ce ne fossero ancora allocati – in questa traccia p ha già esaurito entrambe le clausole) *sia*, soprattutto, quelli per a (la scelta tra le due clausole di a, inclusa l'alternativa a:-r.): quando poi b fallisce, non c'è più modo di ritentare a:-r. – proprio quella scelta è stata tagliata – e l'intero predicato a fallisce. Si risale quindi a g, che ritenta la sua seconda clausola g:-s., anch'essa fallimentare (nessuna clausola per s): il risultato finale è un **fallimento totale**, mentre *senza* il cut la query :-g. avrebbe avuto successo. Questo è un esempio da manuale di come il cut possa cambiare la semantica logica del programma (un "cut rosso", nel gergo standard, anche se questa terminologia non viene usata esplicitamente a lezione).

7.3 Direzionalità del Cut

Il cut taglia le alternative "all'indietro" (la clausola corrente e i goal che lo precedono nel corpo), ma *non* quelle "in avanti" (i goal successivi). Lo mostra questo esempio:

```
a(X,Y) :- b(X),!,c(Y).  
a(0,0).
```

```
b(1). b(2).  
c(1). c(2).
```

```
?- a(X,Y).  
X = 1, Y = 1 ;  
X = 1, Y = 2 ;  
no
```

Il cut, eseguito dopo $b(X)$, taglia sia l'alternativa di clausola $a(0,0)$ per la testa $a/2$, sia l'alternativa $X=2$ (cioè $b(2)$) per il goal $b(X)$ che lo precede – entrambi *prima* del cut. Non taglia invece le alternative di $c(Y)$, che segue il cut: il backtracking su $c(Y)$ produce infatti entrambe le soluzioni $Y=1$ e $Y=2$, con $X=1$ fissato. Risultato: solo due soluzioni, poi **no** – le soluzioni potenziali con $X=2$ o con la clausola $a(0,0)$ restano irraggiungibili.

L'**Answer Set Programming** (ASP) nasce dal dibattito (la "*war of semantics*") su come dare una semantica formale rigorosa alla negazione per fallimento usata dagli interpreti Prolog, ed è oggi il fondamento di un paradigma di programmazione a sé: le soluzioni di un programma ASP sono i suoi **modelli** (*answer set*), non più le *prove* come in Prolog. È particolarmente adatto a problemi combinatori (soddisfacimento di vincoli, planning). Solver diffusi: DLV, smodels, clingo, cmodels.

8.1 ASP vs. Prolog

In ASP l'**ordine dei letterali non ha importanza** (in Prolog conta, per via del backtracking SLD); **Prolog è goal-directed, ASP no** (ASP calcola i modelli stabili dell'intero programma, non risponde a una query specifica); la risoluzione SLD di Prolog può **andare in loop**, mentre gli ASP-solver non lo consentono; **Prolog ha il cut, ASP no**.

8.2 Sintassi

Un programma ASP è un insieme finito di regole

```
a :- b1, ..., bn, not c1, ..., not cm.
```

dove a, b_i, c_j sono letterali della forma p o $\neg p$ ($\neg$ è la **negazione classica**, **not** è la **negazione per fallimento**); la testa a è opzionale, e in sua assenza si ha un **integrity constraint** $\neg a_1, \dots, a_k$, che significa "è inconsistente che $a_1, \dots, a_k$ siano tutti veri" (elimina gli answer set candidati che soddisfano tutto il corpo). ASP si applica formalmente a programmi **ground** (proposizionali); l'uso di variabili è comodo mnemonicamente, ma un programma con variabili deve poter essere ridotto a un numero finito di clausole ground (*grounding*).

La differenza tra le due negazioni è sostanziale: **attraversa :- -treno.** richiede una prova esplicita che il treno *non* sta arrivando (negazione classica, più forte); **attraversa :- not treno.** permette di attraversare semplicemente in assenza di informazione sul treno (chiusura del mondo, negazione per fallimento). Un letterale negato classicamente $\neg p$ non ha proprietà speciali: è trattato come un nuovo atomo positivo distinto da p , e la coerenza tra p e $\neg p$ va imposta esplicitamente con un vincolo $\neg p, \neg p$.

8.3 Semantica: Answer Set come Modello Minimale/Stabile

Un programma ASP **privo di not** ha un unico modello minimale, che è il suo answer set: ad esempio $p:-q.$ $q.$ ha answer set $\{p, q\}$, mentre $p:-q, r.$ $q.$ ha answer set $\{q\}$ (poiché r non è derivabile). In presenza di **not**, si usa il **ridotto** (reduct di Gelfond-Lifschitz) P^S di un programma P rispetto a un insieme di atomi S : (1) si rimuove ogni regola il cui corpo contiene **not** L con $L \in S$; (2) si rimuovono tutti i **not** L residui (con $L \notin S$) dai corpi delle regole rimanenti. Il ridotto P^S , privo di negazione, ha un unico answer set; **se questo coincide con S , allora S è un answer set di P .**

Esempio completo. Programma $p:-a.$ $a:-\text{not } b.$ $b:-\text{not } a.$ Applicando la definizione: la regola $p:-a.$ non ha negazione e resta sempre; la regola $a:-\text{not } b.$ viene *rimossa* se $b \in S$ (passo 1), altrimenti resta come fatto $a.$ (passo 2, il **not** b viene tolto); simmetricamente la regola $b:-\text{not } a.$ viene rimossa se $a \in S$, altrimenti resta come fatto $b.$ Testando tutti gli 8 sottoinsiemi di $\{a, b, p\}$:

S	Ridotto P^S	Answer set M di P^S	$M = S?$
$\emptyset$	$p:-a. \quad a. \quad b. \quad$ (nessuna regola rimossa: $a, b \notin S$)	$\{a, b, p\}$	no
$\{a\}$	$p:-a. \quad a. \quad$ (regola di b rimossa: $a \in S$)	$\{a, p\}$	no
$\{b\}$	$p:-a. \quad b. \quad$ (regola di a rimossa: $b \in S$)	$\{b\}$	sì
$\{p\}$	$p:-a. \quad a. \quad b. \quad$ (nessuna regola rimossa: $a, b \notin S$)	$\{a, b, p\}$	no
$\{a, b\}$	$p:-a. \quad$ (entrambe le regole rimosse: $a, b \in S$)	$\emptyset$	no
$\{a, p\}$	$p:-a. \quad a. \quad$ (regola di b rimossa: $a \in S$)	$\{a, p\}$	sì
$\{b, p\}$	$p:-a. \quad b. \quad$ (regola di a rimossa: $b \in S$)	$\{b\}$	no
$\{a, b, p\}$	$p:-a. \quad$ (entrambe le regole rimosse: $a, b \in S$)	$\emptyset$	no

Solo due candidati superano la verifica $M = S$: $S = \{b\}$ (il ridotto $p:-a. \quad b.$ ha come unico modello minimale $\{b\}$, dato che a non è più un fatto e quindi p non si deriva) e $S = \{a, p\}$ (il ridotto $p:-a. \quad a.$ deriva a come fatto e poi p da a). Risultato: **answer set** = $\{b\}$ e $\{a, p\}$ – coerente con l'intuizione che il programma descrive una scelta esclusiva tra "credere a " (e di conseguenza p) e "credere b ".

Un programma può anche avere un modello classico ma **nessun answer set**: $p:-\text{not } p, \quad d. \quad d.$ ha come modello classico $\{p, d\}$, ma sia $S = \{d\}$ (ridotto $\{p:-d. \quad d.\}$, answer set $\{d, p\} \neq \{d\}$) sia $S = \{p, d\}$ (ridotto $\{d.\}$, answer set $\{d\} \neq \{p, d\}$) falliscono la verifica: nessuno dei due candidati è un answer set. È il caso classico di un **ciclo dispari attraverso la negazione** (*odd loop*), che invalida la stabilità pur ammettendo un modello classico.

8.4 Risolvere Problemi con ASP: Generate-and-Test

ASP risolve un problema con un approccio **generate-and-test**: si descrive un problema caratterizzandone le soluzioni, in modo che ogni answer set del programma sia una soluzione, trovata dal solver nello stesso momento in cui calcola gli answer set. In due fasi: (1) si descrive con delle regole un insieme di **soluzioni candidate** (fase "generate"); (2) si introducono **vincoli** (regole senza testa) che scartano gli answer set non accettabili come soluzione (fase "test").

Un costrutto avanzato utile in questa fase sono gli **aggregati** (*choice/cardinality constraints*):

```
n { p(X) : q(X) } m.
```

seleziona answer set contenenti da n a m atomi p , i cui possibili valori sono presi da q . Ad esempio, per imporre che ogni variabile abbia un solo tipo:

```
1 { ha_tipo(V,T) : tipo(T) } 1 :- variabile(V).
```

Gli esercizi ASP del corso si svolgono con **clingo** (<https://potassco.org/clingo/>), la cui guida utente dettagliata è disponibile sulla pagina Moodle del corso.

Parte II

Planning, Sistemi a Regole e Incertezza (Prof. Roberto Micalizio)

9 Fondamenti del Planning Automatico

Il **planning** (pianificazione automatica) è, nelle parole di Patrik Haslum, "l'arte e la pratica di pensare prima di agire"; più operativamente (Jörg Hoffmann), è la selezione di un corso d'azione che conduce a un obiettivo, a partire da una descrizione ad alto livello del mondo. È un **processo deliberativo** che sceglie e organizza le azioni in base all'effetto atteso, e l'*AI planning* ne studia la calcolabilità.

9.1 Il Modello Concettuale: State Transition System

Un dominio di pianificazione si modella con uno **State Transition System** (STS):

$$\Sigma = (S, A, E, \gamma)$$

dove S è un insieme finito di **stati**, A un insieme finito di **azioni**, E un insieme finito di **eventi** (accadimenti fuori dal controllo dell'esecutore), e $\gamma : S \times (A \cup E) \rightarrow 2^S$ è la **relazione di transizione di stato**. Un'azione a è *applicabile* in s se $\gamma(s, a) \neq \emptyset$; applicarla causa una transizione a un $s' \in \gamma(s, a)$. Un STS è naturalmente un **grafo diretto** $G = (S, E_G)$, con un arco $s \rightarrow s'$ etichettato $u \in A \cup E$ se $s' = \gamma(s, u)$: un **piano**, in prima approssimazione, è un cammino da uno stato iniziale I a un goal G in questo grafo.

L'esecuzione reale di un piano richiede un'architettura a due componenti: un **Planner**, che dati Σ , lo stato iniziale e il goal genera un piano; e un **Controller**, che dato il piano e lo stato corrente (osservato tramite una funzione $\eta : S \rightarrow O$) seleziona ed esegue un'azione. Nello schema più semplice si assume che gli eventi non interferiscano con le azioni del controller; nel **continual planning**, più realistico, pianificazione ed esecuzione si intrecciano (*plan supervision*, *plan revision/diagnosis*, *re-planning*) proprio perché il mondo reale può divergere dal modello Σ usato per pianificare.

9.2 Le Assunzioni del Planning Classico

Il **planning classico** adotta otto assunzioni semplificative (A0–A7), ciascuna rilassabile a costo di una maggiore complessità:

Assunzione	Definizione	Conseguenze del rilassamento
A0 – Dominio finito	Σ ha un numero finito di stati	problemi di decidibilità/terminazione
A1 – Osservabilità completa	η è la funzione identità	osservazioni ambigue $\Rightarrow$ <i>belief state</i> ; <i>conformant planning</i>
A2 – Determinismo	$ \gamma(s, u) \leq 1$ per ogni s, u	piani con rami condizionali; <i>conditional planning</i> con sensing actions
A3 – Staticità	$E = \emptyset$	mondo non deterministico dal punto di vista del planner; <i>continual planning</i>
A4 – Goal semplici	goal = stato/insieme di stati da raggiungere	vincoli su piani, funzioni di utilità/costo
A5 – Piani sequenziali	il piano è una sequenza totalmente ordinata	azioni parallele, strutture più complesse
A6 – Tempo implicito	azioni/eventi istantanei	azioni durative, concorrenza, deadline
A7 – Single agent	un solo planner e un solo controller	<i>multi-agent planning</i> : coordinazione, negoziazione

9.3 Il Problema di Planning Classico

Sotto le assunzioni A0–A7, un **problema di pianificazione classica** è la tripla

$$P = (\Sigma, s_0, S_g)$$

con $\Sigma = (S, A, \gamma)$ (niente eventi, per A3), $s_0 \in S$ stato iniziale, $S_g \subset S$ insieme di stati goal. Una **soluzione** π è una sequenza totalmente ordinata di azioni ground $\pi = \langle a_1, \dots, a_n \rangle$ che genera una sequenza di transizioni $\langle s_0, s_1, \dots, s_n \rangle$ con $s_1 = \gamma(s_0, a_1)$, $s_k = \gamma(s_{k-1}, a_k)$ per $k = 2, \dots, n$, e $s_n \in S_g$.

Restano aperte tre grandi domande, che guidano il resto di questa parte: (1) come rappresentare stati e azioni senza dover enumerare esplicitamente S, A, γ ? (2) come cercare una soluzione in modo efficiente, con quali algoritmi ed euristiche? (3) come generalizzare le soluzioni a contesti più realistici, rilassando A0–A7?

9.4 Complessità

Il planning classico è computazionalmente costoso. Si definiscono due problemi decisionali: **PlanSAT** (esiste un piano soluzione?) e **Bounded PlanSAT** (esiste un piano di lunghezza $\leq k$?). Per il planning classico entrambi sono **decidibili** (spazio di ricerca finito); estendendo il linguaggio con simboli di funzione lo spazio diventa infinito, PlanSAT diventa solo **semidecidibile** mentre Bounded PlanSAT resta decidibile. In generale, PlanSAT e Bounded PlanSAT sono in **PSPACE**, e in molti casi pratici Bounded PlanSAT è **NP-completo** mentre PlanSAT è in **P**: trovare *una* soluzione è quindi tipicamente meno costoso che trovarne una *ottima* – da cui la necessità di euristiche, possibilmente *domain-independent*.

9.5 Proprietà di un Buon Algoritmo di Pianificazione

Un pianificatore è **sound** (corretto) se ogni soluzione trovata è realmente eseguibile: tutti i goal sono soddisfatti, nessuna preconditione resta *open* (non fornita da s_0 o da un'azione precedente), nessun vincolo è violato. È **completo** se trova sempre una soluzione quando il problema è risolubile, ed è **strettamente completo** se nessuna operazione di pruning scarta soluzioni valide. È **ottimo** se l'ordine con cui trova le soluzioni è coerente con una misura di qualità del piano (lunghezza, costo).

10 Rappresentazione a Variabili di Stato e Ricerca nello Spazio degli Stati

10.1 State-Variable Representation

Nota: questo formalismo con terminologia **rigid/varying** è quello del testo di riferimento (Ghallab, Nau, Traverso, *Automated Planning*) più che delle slide del corso, che usano direttamente la notazione insiemistica precondizioni/effetti positivi e negativi ripresa nella sottosezione successiva (Progression) e in STRIPS. È comunque un buon modo per capire in astratto perché convenga sostituire una logica del primo ordine (FOL) completa – che si vedrà formalizzata più avanti, in Sezione 11 – con una rappresentazione più snella: per evitare di enumerare esplicitamente S, A, γ , conviene una **rappresentazione a variabili di stato**. Dato l'insieme B degli oggetti rilevanti del dominio, le proprietà si distinguono in **rigid** (relazioni invarianti, es. `adjacent(d1, d2)`, rappresentate come costanti booleane) e **varying** (fluenti che cambiano per effetto delle azioni, es. `loc(r1)`, rappresentate come **variabili** cui si assegna un valore – analogia OO: equivale a scrivere `r1.loc`). Un insieme prefissato di variabili di stato $X = \{loc(r_1), loc(r_2), cargo(r_1), \dots\}$, ciascuna con un dominio (*Range*), definisce lo spazio: uno **stato** s_i è semplicemente una funzione $X \rightarrow \text{valori}$, equivalentemente un insieme di coppie (x, w) .

Uno **schema di azione** specifica precondizioni come relazioni costanti ($rel(t_1, \dots, t_k)$ o la sua negazione) o test su variabili di stato ($sv(t_1, \dots, t_k) = t_0$ o $\neq t_0$), ed effetti come assegnamenti $sv(t_1, \dots, t_k) \leftarrow t_0$:

```
move(r,l,m) take(r,l,c)
pre: loc(r)=l, adjacent(l,m) pre: cargo(r)=nil, loc(r)=l, loc(c)=l
eff: loc(r) <- m eff: cargo(r) <- c, loc(c) <- r
```

La transizione di stato per un'azione applicabile è $\gamma(s, a) = s'$, con $s' = \{(x, w) \mid x \leftarrow w \in \text{eff}(a)\} \cup \{(x, w) \in s \mid x \text{ non occorre in } \text{eff}(a)\}$. La rappresentazione FOL classica (predicati) e quella a variabili di stato sono **equivalenti** (stesso potere espressivo, $P(b_1, \dots, b_k) \rightsquigarrow x_P(b_1, \dots, b_k) = \text{true}$ e viceversa); quella a variabili di stato è però più sintetica e si presta meglio a estensioni non classiche (osservabilità parziale, fluenti numerici, euristiche).

10.2 Progression: Ricerca in Avanti

Un'azione ground a è applicabile in s se ogni precondizione positiva è in s e ogni precondizione negativa non lo è; la transizione (con precondizioni ed effetti espressi in termini insiemistici $\text{effects}^+/\text{effects}^-$) è

$$\gamma(s, a) = (s \setminus \text{effects}^-(a)) \cup \text{effects}^+(a).$$

La **progression** (ricerca in avanti) parte da s_0 e applica iterativamente azioni applicabili fino a raggiungere uno stato che soddisfa il goal:

```
function fwdSearch(0, s0, sg)
  state <- s0
  pi <- <>
  loop
    if state.satisfies(sg) then return pi
    applicables <- {istanze ground da 0 applicabili in state}
    if applicables.isEmpty() then return failure
    action <- applicables.chooseOne() % scelta non deterministica esaustiva
    state <- gamma(state, action)
    pi <- pi . <action>
```

```
return failure
```

fwdSearch è **sound** (se termina con un piano, è una soluzione valida) e **completo** (se esiste una soluzione, esiste una traccia che la trova). Il vantaggio è l'estrema intuitività dell'implementazione.

10.3 Regression: Ricerca all'Indietro

Il numero di azioni applicabili a uno stato è spesso molto grande, con branching factor elevato: la **regression** attacca il problema partendo dal goal e regredendo all'indietro. Il **regresso** di un goal g attraverso un'azione ground a tale che $g_i \in effects^+(a)$ per qualche $g_i \in g$ è

$$g' = \gamma^{-1}(g, a) = (g \setminus effects^+(a)) \cup pre(a)$$

(gli effetti negativi non intervengono nel calcolo del predecessore). Pseudocodice:

```
function bwdSearch(0, s0, g)
  subgoal <- g
  pi <- <>
  loop
    if s0.satisfies(subgoal) then return pi
    relevants <- {istanze ground da 0 rilevanti per subgoal}
    if relevants.isEmpty() then return failure
    action <- relevants.chooseOne()
    subgoal <- gamma_inv(subgoal, action)
    pi <- <action> . pi
  return failure
```

Un'azione è **rilevante** per un sottogoal se produce almeno uno degli atomi del goal e non ne nega nessuno. Il vantaggio della regression è un fattore di ramificazione più contenuto; lo svantaggio è che, per gestire correttamente più stati goal candidati simultaneamente (*belief state*), le strutture dati diventano più costose. Nonostante il vantaggio teorico, **in pratica la ricerca in avanti è preferita**.

Progression	Regression
Parte da un singolo stato iniziale	Parte da un insieme di stati goal
Un operatore genera un unico stato successore	Un passo all'indietro può avere più predecessori
Spazio di ricerca $\approx$ spazio degli stati	Ogni stato di ricerca è un insieme di stati del dominio

Alcuni pianificatori reali, classificati secondo direzione/rappresentazione/algoritmo/tecnica di controllo: **FF** (Hoffmann & Nebel, 2001, avanti, esplicito, hill-climbing, euristica inammissibile con pruning, subottimo); **LAMA** (Richter & Westphal, 2008, avanti, variante di A*, euristica FF + landmark non ammissibile, subottimo); **Fast Downward** (Helmert et al., 2011, avanti, A*, euristiche ammissibili come LM-cut e merge-and-shrink, ottimo).

STRIPS (Stanford Research Institute Problem Solver, Fikes & Nilsson 1971) introduce una rappresentazione esplicita degli operatori di pianificazione, operazionalizzando la distinzione tra stati, sotto-goal e applicazione di un operatore, e gestendo il *frame problem*. Si basa su due idee: il **linear planning** (risolvere un goal alla volta, tramite uno **stack dei goal**, passando al successivo solo quando il precedente è raggiunto) e la **means-end analysis** (analizzare la differenza tra stato corrente e goal, e trovare un operatore che la riduce, ripetendo l'analisi sui sotto-goal per regressione).

11.1 Stati e Operatori

Il linguaggio STRIPS è un frammento di FOL: numero finito di predicati e costanti, **senza** simboli di funzione né quantificatori. Uno stato è una congiunzione di **atomi ground**, sotto la **closed world assumption** (assunzione di mondo chiuso): un atomo vale in s se e solo se vi compare esplicitamente. Le relazioni si distinguono in **fluente** (possono cambiare, es. `On`, `OnTable`) e **persistenti** (*state invariant*, es. `Block()`, mai modificate da alcuna azione).

Un **plan operator** è la tripla $o : (name(o), precondition(o), effects(o))$, dove $precondition(o)$ si divide in $precondition^+(o)$ e $precondition^-(o)$, ed $effects(o)$ in $effects^+(o)$ (l'**add-list**) ed $effects^-(o)$ (la **delete-list**); una variabile può comparire negli effetti solo se già presente nelle precondizioni. Esempio nel mondo dei blocchi:

```
Pick_Block(?b, ?c)
Pre: Block(?b), Handempty, Clear(?b), On(?b,?c), Block(?c)
Eff: Holding(?b), Clear(?c), not(Handempty), not(On(?b,?c))

Put_Block(?b, ?c)
Pre: Block(?b), Holding(?b), Clear(?c), Block(?c)
Eff: not(Holding(?b)), not(Clear(?c)), Handempty, On(?b,?c)
```

(un'azione STRIPS è un'istanziatura ground di un plan operator; questi due soli operatori, richiedendo entrambi `Block(?c)` tra le precondizioni, non permettono di interagire col tavolo – serve estenderli con operatori dedicati a prelevare/posare un blocco sul tavolo, come si vede nell'esempio completo sotto).

11.2 L'Algoritmo STRIPS

```
STRIPS(initState, goals)
  state = initState; plan = []; stack = []
  Push goals on stack
  Repeat until stack is empty
    - Se in cima allo stack c'e' un goal g gia' soddisfatto in state -> pop
    - Se in cima c'e' un goal congiuntivo g -> scegli un ordine per i
      sotto-goal e mettili sullo stack (linearizzazione)
    - Se in cima c'e' un goal atomico sg -> scegli un operatore o la cui
      effects+ soddisfa sg (means-end analysis); sostituisci sg con o;
      metti le precondizioni di o sullo stack
    - Se in cima c'e' un operatore o completamente istanziato -> esegui
      o (state = apply(o,state), progression) e aggiungilo al piano
```

11.3 Esempio Completo: Blocks World

Goal $On(A, C) \wedge On(C, B)$, con stato iniziale (torre C su A, B separato)

$Clear(B), Clear(C), On(C, A), On(A, Table), On(B, Table), Handempty.$

Traccia (con **Pick**/**Put** per prelevare/posare su un blocco, **PickT**/**PutT** per prelevare/posare sul tavolo): si sceglie di risolvere prima $On(C, B)$: serve $Holding(C)$, ottenuto con **Pick(C)** (le sue precondizioni, incluso $On(C, A)$, sono già vere); si esegue quindi **Put(C, B)**. Resta $On(A, C)$: serve $Holding(A)$, ma A è sul tavolo, quindi si usa **PickT(A)** (precondizioni verificate) seguito da **Put(A, C)**. Piano finale:

$$\pi = [\text{Pick}(C); \text{Put}(C, B); \text{PickT}(A); \text{Put}(A, C)].$$

11.4 Limiti del Linear Planning

Lo STRIPS planner, essendo *lineare*, è sound e ha un vantaggio di ricerca notevole quando i goal sono indipendenti, ma **può produrre piani subottimi e non è completo**.

Subottimalità – Sussman’s Anomaly. Con goal $On(A, B) \wedge On(B, C)$ (stesso stato iniziale di prima) e ordine "prima $On(A, B)$ ": per rendere A prendibile occorre liberarlo da C (**Pick(C, A)**, **PutT(C)**), poi **PickT(A)**, **Put(A, B)** – ma per il secondo goal $On(B, C)$ serve $Holding(B)$, e B non è più libero perché A vi è appena stato posato: occorre **disfare** $On(A, B)$ (**Pick(A, B)**, **PutT(A)**), completare $On(B, C)$ (**PickT(B)**, **Put(B, C)**), e infine **ripristinare** $On(A, B)$ (**PickT(A)**, **Put(A, B)**): un piano di 10 azioni invece delle 4 dell’esempio precedente, per l’interdipendenza dei due sotto-goal combinata con un ordinamento sfavorevole.

Incompletezza. Dominio logistico con **Load/Unload/Fly**, dove **Fly** consuma **HaveFuel** (effetto non reversibile): stato iniziale con due pacchi e un aereo a $LocA$, goal entrambi a $LocB$. Risolvendo linearmente il primo pacco (**Load**, **Fly**, **Unload**), il carburante si esaurisce e il planner lineare, non tornando mai sui propri passi, **non riesce più a raggiungere il secondo goal** – un’incompletezza che nasce direttamente dalla non reversibilità di alcune azioni in certi domini.

PDDL (Planning Domain Definition Language) è lo standard de facto per l'input dei pianificatori nelle competizioni internazionali; PDDL 1.0 coincide sostanzialmente col linguaggio STRIPS. Un task di pianificazione è definito da **oggetti**, **predicati**, **stato iniziale**, **specifiche del goal** e **azioni/operatori**, separati in due file: il **domain file** (predicati e azioni, conoscenza indipendente dal problema) e il **problem file** (oggetti, stato iniziale, goal, conoscenza specifica del problema):

```
(define (domain <domain-name>)
  <predicati>
  <azione 1> ... <azione N>)

(define (problem <problem-name>)
  (:domain <domain-name>)
  <oggetti> <stato iniziale> <goal>)
```

12.1 Esempio Completo: Gripper

Un robot con due bracci sposta palloni tra due stanze. Predicati: ROOM, BALL, GRIPPER, at-robby(x), at(b,r), free(g), carry(o,g). Azioni move, pick, drop:

```
(define (domain gripper-strips)
  (:predicates (room ?r) (ball ?b) (gripper ?g) (at-robby ?r)
               (at ?b ?r) (free ?g) (carry ?o ?g))
  (:action move
   :parameters (?from ?to)
   :precondition (and (room ?from) (room ?to) (at-robby ?from))
   :effect (and (at-robby ?to) (not (at-robby ?from))))
  (:action pick
   :parameters (?obj ?room ?gripper)
   :precondition (and (ball ?obj) (room ?room) (gripper ?gripper)
                     (at ?obj ?room) (at-robby ?room) (free ?gripper))
   :effect (and (carry ?obj ?gripper) (not (at ?obj ?room))
              (not (free ?gripper))))
  (:action drop
   :parameters (?obj ?room ?gripper)
   :precondition (and (ball ?obj) (room ?room) (gripper ?gripper)
                     (carry ?obj ?gripper) (at-robby ?room))
   :effect (and (at ?obj ?room) (free ?gripper)
              (not (carry ?obj ?gripper)))))
```

```
(define (problem strips-gripper2)
  (:domain gripper-strips)
  (:objects rooma roomb ball1 ball2 ball3 ball4 left right)
  (:init
   (room rooma) (room roomb)
   (ball ball1) (ball ball2) (ball ball3) (ball ball4)
   (gripper left) (gripper right)
   (at-robby rooma) (free left) (free right)
   (at ball1 rooma) (at ball2 rooma)
   (at ball3 rooma) (at ball4 rooma))
  (:goal (and (at ball1 roomb) (at ball2 roomb)
              (at ball3 roomb) (at ball4 roomb))))
```

Leggendo le tre azioni riga per riga: **move** ha come unico parametro le due stanze coinvolte; la preconditione richiede che entrambe siano stanze valide e che il robot sia attualmente in **?from**; l'effetto è la coppia **add/delete** tipica di STRIPS, scritta esplicitamente con **and/not**: (**at-robby ?to**) è l'**effetto positivo** (aggiunto allo stato), (**not (at-robby ?from)**) è l'**effetto negativo** (rimosso dallo stato) – PDDL non ha "liste" add/delete separate come STRIPS classico, ma codifica la stessa informazione con la presenza o meno di **not** davanti a ciascun letterale dell'effetto. Analogamente, **pick** richiede che il pallone e il robot siano nella stessa stanza e che il gripper sia libero (**free ?gripper**); l'effetto aggiunge (**carry ?obj ?gripper**) e rimuove sia (**at ?obj ?room**) (il pallone non è più a terra) sia (**free ?gripper**) (il gripper è ora occupato). **drop** è l'azione simmetrica: richiede di avere già in mano il pallone (**carry ?obj ?gripper**) ed essere nella stanza di destinazione, e nell'effetto scambia esattamente add e delete rispetto a **pick** (il pallone torna a terra, il gripper torna libero, la presa viene rimossa).

12.2 Requirements

PDDL è cresciuto in complessità nel tempo: la clausola (**:requirements ...**) dichiara quali estensioni del linguaggio sono usate, così un planner può verificare se è in grado di risolvere il problema (o se può limitarsi al frammento "core"). Le principali:

Requirement	Significato
:strips	effetti add/delete in stile STRIPS di base
:typing	tipi nelle dichiarazioni di variabili
:disjunctive-preconditions	or nelle preconditioni
:equality	predicato built-in =
:existential/universal-preconditions	exists/forall nelle preconditioni
:conditional-effects	when negli effetti
:negative-preconditions	not diretto nelle preconditioni (senza, si assume closed-world)
:fluents	funzioni/valori numerici (usati dai costi e dalle risorse)
:domain-axioms	assiomi di dominio, necessari per dare semantica agli effetti condizionali con forall
:adl	unione di strips, typing, disjunctive, equality, quantified, conditional

Queste sono le più usate in pratica; lo standard ne definisce altre più specialistiche (**:action-expansions**, **:foreach-expansions**, **:safety-constraints**, **:open-world**, **:true-negation**, ...), in genere legate a estensioni storiche poco diffuse nei planner moderni.

12.3 Tipi e Gerarchie

Sono predefiniti i tipi **object** e **number**; si possono definire gerarchie con radice uno dei due, es. (**:types integer float - number physob**). Il dominio Gripper tipizzato (che richiede **:typing**):

```
(define (domain gripper-typed)
  (:requirements :typing)
  (:types room ball gripper)
  (:constants left right - gripper)
  (:predicates (at-robby ?r - room) (at ?b - ball ?r - room)
               (free ?g - gripper) (carry ?o - ball ?g - gripper))
  (:action move
    :parameters (?from ?to - room)
    :precondition (at-robby ?from)
    :effect (and (at-robby ?to) (not (at-robby ?from)))))
```

con vincoli di tipo '**?from ?to - room**' nei parametri che sostituiscono il controllo esplicito del predicato '**ROOM(x)**' visto nella versione non tipizzata.

12.4 Effetti Condizionali e Quantificatori

Il dominio logistico ad ADL mostra entrambi i costrutti avanzati insieme – **forall** nell'effetto, con un **when** annidato, per spostare *tutti* i pacchi contenuti in un camion quando questo si sposta:

```
(:action drive-truck
:parameters (?truck - truck ?loc-from ?loc-to - location ?city - city)
:precondition (and (at ?truck ?loc-from) (in-city ?loc-from ?city)
                  (in-city ?loc-to ?city))
:effect (and (at ?truck ?loc-to) (not (at ?truck ?loc-from))
            (forall (?x - obj)
              (when (in ?x ?truck)
                (and (not (at ?x ?loc-from))
                     (at ?x ?loc-to))))))
```

Il costrutto (**forall** (?x - obj) EFFETTO) itera concettualmente su *tutti* gli oggetti di tipo **obj** esistenti nel problema, applicando EFFETTO a ciascuno; da solo, però, applicherebbe l'effetto anche ai pacchi che non sono sul camion, il che sarebbe errato. Per questo è annidato dentro (**when** CONDIZIONE EFFETTO): la condizione (**in** ?x ?truck) viene valutata nello stato *prima* dell'esecuzione dell'azione (non in quello risultante), e l'effetto tra parentesi si applica a ?x solo se la condizione era vera per quel particolare oggetto. La combinazione **forall+when** realizza quindi "per ogni pacco che si trova nel camion (e solo per quelli), spostalo insieme al camion" – senza **when** il quantificatore da solo non potrebbe filtrare gli oggetti da un insieme più ampio.

12.5 Due Problemi di Modellazione Ricorrenti

Nel mondo dei blocchi in PDDL, esprimere "*b* non ha nulla sopra" richiederebbe in FOL un quantificatore ($\neg \exists x On(x, b)$); PDDL di base non ha quantificatori, e la soluzione standard è introdurre esplicitamente un predicato **Clear**(*b*) mantenuto dagli effetti delle azioni. Un secondo problema nasce trattando il tavolo come un oggetto ordinario: **Clear**(*Table*) o richiederlo come preconditione di **Move** non ha senso (il tavolo non "si libera" mai davvero); la soluzione standard è introdurre un operatore dedicato **MoveToTable**(*b*, *x*) che tratta il tavolo come caso speciale, non come parametro generico dell'operatore **Move**. Lo stesso genere di problema si ripresenta nel dominio logistico (Figure 10.1 di AIMA): serve distinguere **In**(*c*, *p*) (un carico a bordo di un aereo) da **At**(*x*, *a*) (un oggetto – aereo o carico – in un aeroporto), e va vincolato esplicitamente **from≠to** in **Fly** per evitare voli spuri verso lo stesso aeroporto.

12.6 Estensioni di PDDL

Oltre a quanto visto: gli **effetti condizionali** (**when** <condizione> <effetto>); le **azioni durative** (:durative-actions), suddivise in evento iniziale, evento finale e condizione invariante per tutta la durata; i **fluenti numerici**, utili per tracciare il consumo di risorse; i **quantificatori** (**forall** (?v...) ...) e (**exists** (?v...) ...).

Formulare il planning come un **CSP** (Constraint Satisfaction Problem) permette di **ritardare le decisioni** finché non sono strettamente necessarie: è il **principio least-commitment**, applicato a due tipi di scelte rimandabili – gli **ordinamenti** tra azioni (non ordinarle finché non serve) e i **bindings** delle variabili (non vincolarle a costanti finché non serve). Ne nasce la **ricerca nello spazio dei piani** (*Plan-Space Planning*, PSP): ogni nodo di ricerca è un **piano parzialmente ordinato** con dei **difetti** (*flaws*); ad ogni passo si rimuove un difetto con un raffinamento incrementale. Se l'algoritmo termina con successo, il piano risultante è completamente istanziato ma solo *parzialmente* ordinato.

13.1 Formalizzazione del Piano Parziale

Un piano parziale è la quadrupla $\pi = \langle A, O, B, L \rangle$: A è l'insieme di azioni (anche solo parzialmente istanziate), che include sempre un'azione fittizia a_0 (senza precondizioni, con $effects^+(a_0) \equiv s_0$) e a_∞ (senza effetti, con $pre(a_\infty) \equiv s_{goal}$); O è un insieme di vincoli di ordinamento ($a_i \prec a_j$), solo parzialmente definito; B è un insieme di vincoli sulle variabili ($v_i = C, v_i \neq C, v_i = v_j, v_i \neq v_j$); L è un insieme di **causal link** $a_i \xrightarrow{p} a_j$, che registrano che un effetto di a_i soddisfa la precondizione p di a_j .

13.2 Flaws: Open Goal

Una precondizione p di un'azione b è un **open goal** se manca un causal link a suo supporto. Per risolverla: si trova un'azione a (già in π , o da aggiungere) tale che p possa appartenere a $effects^+(a)$ e a possa precedere b ; la si istanzia al minimo indispensabile (least-commitment); si aggiunge $a \prec b$ a O ; si aggiunge $a \xrightarrow{p} b$ a L .

13.3 Flaws: Threat

Dato un causal link $l : a \xrightarrow{p} b \in L$, un'azione c (detta *clobberer*) **minaccia** l se può modificare il valore di verità di p posizionandosi tra a e b . Tre modi per risolvere la minaccia:

- **Promotion**: impone $c \prec a$ (anticipa c prima di a);
- **Demotion**: impone $b \prec c$ (posticipa c dopo b);
- **Separation**: impone un vincolo di non-codesignazione (es. $x \neq Block1$) che impedisce all'effetto di c di unificare con p .

13.4 Algoritmo PSP

```
PSP(Sigma, pi) % pi = <A,O,B,L>
  Flaws(pi) <- OpenGoals(pi) unione Threats(pi)
  if Flaws(pi) = vuoto then return pi
  scegli un f in Flaws(pi)
  R <- {tutti i resolver possibili per f}
  if R = vuoto then return failure
  scegli (non deterministicamente) un rho in R
  pi' <- rho(pi)
  return PSP(Sigma, pi')
```

Si parte da $\pi^1 = (\langle a_0, a_\infty \rangle, \{a_0 \prec a_\infty\}, \emptyset, \emptyset)$. **PSP è sound e completo**: restituisce un piano parzialmente ordinato tale che *qualunque* ordinamento totale delle sue azioni raggiunge il goal; dove il dominio lo consente, le azioni non strettamente sequenziali possono essere eseguite in parallelo.

13.5 Esempio Completo: Scambio di Posizione tra Due Robot

Due robot r_1, r_2 devono scambiarsi di posizione tra tre depositi d_1, d_2, d_3 , con l'unica azione $\text{move}(\mathbf{r}, \mathbf{d}, \mathbf{d}')$, preconditione $\text{loc}(r)=d \wedge \text{occupied}(d')=\text{nil}$ ed effetto

$$\text{loc}(r) \leftarrow d', \quad \text{occupied}(d) \leftarrow \text{nil}, \quad \text{occupied}(d') \leftarrow r.$$

Stato iniziale e goal:

$$\begin{aligned} s_0 &= \{\text{loc}(r_1)=d_1, \text{loc}(r_2)=d_2, \text{occupied}(d_1)=T, \text{occupied}(d_2)=T, \text{occupied}(d_3)=F\} \\ g &= \{\text{loc}(r_1)=d_2, \text{loc}(r_2)=d_1\} \end{aligned}$$

Si parte da $a_0 \rightarrow a_\infty$. Per risolvere i due open goal di a_∞ si introducono $a_1=\text{move}(r_1, d, d_2)$ e $a_2=\text{move}(r_2, d', d_1)$ (con d, d' ancora libere). Risolvendo gli open goal di a_1 : $\text{loc}(r_1)=d$ si unifica con l'effetto di a_0 ($d=d_1$); ma $\text{occupied}(d_2)=\text{nil}$ non è soddisfatta da a_0 (che ha $\text{occupied}(d_2)=r_2$), quindi si introduce $a_3=\text{move}(r, d_2, d'')$. Risolvendo gli open goal di a_2 : $\text{loc}(r_2)=d'$ si unifica con l'effetto di a_3 ($r=r_2, d''=d'$). Risolvendo infine gli open goal di a_3 : $\text{occupied}(d_3)=\text{nil}$ è soddisfatta da a_0 (unico deposito libero), quindi $d'=d_3$; con questo, $a_3=\text{move}(r_2, d_2, d_3)$ e $a_2=\text{move}(r_2, d_3, d_1)$ (con preconditione $\text{occupied}(d_1)=\text{nil}$ ora soddisfatta dall'effetto di a_1). Il piano finale, parzialmente ordinato $a_0 \prec a_3 \prec a_1 \prec a_2 \prec a_\infty$:

$$a_3=\text{move}(r_2, d_2, d_3) \rightarrow a_1=\text{move}(r_1, d_1, d_2) \rightarrow a_2=\text{move}(r_2, d_3, d_1)$$

(r_2 si sposta prima nel deposito libero d_3 , liberando d_2 per r_1 ; poi r_2 si sposta da d_3 nel deposito ormai libero d_1).

13.6 Glossario

Un piano è **totalmente ordinato** se O ordina ogni coppia di azioni; è **parzialmente ordinato** se, pur non essendolo, è privo di difetti (ogni sua linearizzazione è una soluzione valida); è un **piano parziale** se contiene ancora difetti e/o azioni non completamente istanziate. Un **open goal** è una preconditione non ancora supportata da alcun causal link; un **threat** è un'azione che minaccia un causal link esistente.

Il **planning graph** (GP) è una struttura dati, costruibile in **tempo polinomiale**, utile sia per definire euristiche *domain-independent* sia per estrarne direttamente un piano (algoritmo GraphPlan). È un grafo **orientato, aciclico, organizzato a livelli** che si alternano: un livello di **proposizioni** S_i e uno di **azioni** A_i . S_0 contiene un nodo per ogni fluente vero nello stato iniziale; A_0 contiene le azioni ground applicabili a S_0 più una speciale **azione no-op** per ogni proposizione (persistenza); ogni S_i contiene tutti i letterali che *potrebbero* valere al tempo i (anche P e $\neg P$ insieme: ogni S_i è un *belief state*) e ogni A_i tutte le azioni le cui precondizioni *potrebbero* essere soddisfatte al tempo i . Il grafo cresce monotonicamente e prima o poi **si livella** (due livelli consecutivi diventano identici).

14.1 Mutua Esclusione (Mutex)

Il GP traccia un sottoinsieme delle interazioni negative tramite link di **mutua esclusione**. Tra due azioni in uno stesso A_i : **effetti inconsistenti** (una nega gli effetti dell'altra); **interferenza** (l'effetto di una nega una precondizione dell'altra); **competizione delle precondizioni** (una precondizione di una è mutex con una precondizione dell'altra). Tra due letterali in uno stesso S_i : **complementarità** (uno è la negazione dell'altro); **supporto inconsistente** (ogni coppia di azioni di A_{i-1} che li producono è mutex). I mutex tra letterali possono *sparire* a livelli successivi (proprietà di monotonicità): un caso tipico è quando emerge una seconda via, non in conflitto, per produrre uno dei due letterali.

Per rendere concrete le tre condizioni tra azioni, si anticipi il dominio "have-cake" usato nell'esempio della sezione successiva (**Eat(Cake)**, precondizione *Have(Cake)*, effetti $\neg\text{Have(Cake)} \wedge \text{Eaten(Cake)}$; **Bake(Cake)**, precondizione $\neg\text{Have(Cake)}$, effetto *Have(Cake)*; più la no-op di persistenza per ogni fluente, con precondizione ed effetto identici al fluente stesso). In A_1 , la coppia **Eat(Cake)** e la **no-op di persistenza** di *Have(Cake)* (precondizione ed effetto *Have(Cake)*) è mutex per *due* motivi contemporaneamente: **effetti inconsistenti** (l'effetto di **Eat(Cake)** è $\neg\text{Have(Cake)}$, quello della no-op è *Have(Cake)*: effetti direttamente opposti sullo stesso fluente) e, equivalentemente, **interferenza** (l'effetto $\neg\text{Have(Cake)}$ di **Eat(Cake)** nega proprio la precondizione *Have(Cake)* richiesta dalla no-op) – in domini così piccoli le due condizioni spesso coincidono sulla stessa coppia di azioni, perché la no-op ha precondizione ed effetto uguali. La terza condizione, **competizione delle precondizioni**, si vede invece sulla coppia **Eat(Cake)/Bake(Cake)**: la precondizione di **Eat(Cake)** è *Have(Cake)*, quella di **Bake(Cake)** è $\neg\text{Have(Cake)}$ – sono l'una la negazione dell'altra, dunque mutex tra loro in S_0 , il che rende mutex anche le due azioni in A_1 .

La costruzione ha complessità $O(n(a+l)^2)$, con l numero di letterali, a numero di azioni, n numero di livelli: ogni S_i ha $\leq l$ nodi e $\leq l^2$ mutex, ogni A_i ha $\leq (l+a)$ nodi e $\leq (a+l)^2$ mutex.

14.2 GraphPlan come Euristiche

Se un letterale del goal non compare mai nel GP, il goal è certamente irraggiungibile; se invece tutti compaiono, il problema *potrebbe* essere risolubile (non è garantito). Da qui euristiche per il costo di un letterale (profondità del suo primo livello di comparsa, ammissibile ma poco precisa) e per una congiunzione di letterali: **max-level** (il massimo dei livelli, poco preciso); **somma dei livelli** (assume indipendenza, **non ammissibile**); **set-level** (il primo livello in cui tutti i letterali compaiono *senza* mutex tra loro a coppie – **ammissibile**, ma ignora dipendenze tra tre o più letterali).

14.3 L'Algoritmo GraphPlan

Teorema: se esiste un piano valido, questo è un sottografo del planning graph.

```
function GRAPHPLAN(problem) returns solution or failure
  graph <- INITIAL-PLANNING-GRAPH(problem)
  goals <- CONJUNCTS(problem.GOAL)
  nogoods <- tabella hash vuota
  for tl = 0 to infinito do
    if goals tutti non-mutex in Stl del grafo then
      solution <- EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)
      if solution != failure then return solution
    if graph e nogoods si sono entrambi livellati then return failure
  graph <- EXPAND-GRAPH(graph, problem)
```

Extract-Solution cerca un piano con una ricerca backward su un sottografo AND/OR del GP (rami OR: le azioni che producono un letterale goal; rami AND: le precondizioni di un'azione scelta), tramite due funzioni mutuamente ricorsive:

```
Extract-Solution(G, g, i)
  if i = 0 then return <>
  if g in no-good(i) then return failure
  pi <- GP-Search(G, g, {}, i-1)
  if pi != failure then return pi
  no-good(i) <- no-good(i) unione {g}
  return failure

GP-Search(G, g, pi_i, i)
  if g = {} then
    Pi <- Extract-Solution(G, unione{precond(a) : a in pi_i}, i)
    if Pi = failure then return failure
    return Pi unione pi_i
  else
    scegli un p in g
    resolvers <- {a in Ai : p in effects+(a), nessun b in pi_i e' mutex con a}
    if resolvers = {} then return failure
    scegli (non det.) un resolver a per p
    return GP-Search(G, g - effects+(a), pi_i unione {a}, i)
```

Ogni fallimento di **Extract-Solution** su una coppia (L, i) viene ricordato nella tabella **no-good**: un meccanismo di **memoization** che evita di ripetere ricerche già fallite.

14.4 Esempio: "Have Cake and Eat It Too"

```
Init(Have(Cake))
Goal(Have(Cake) and Eaten(Cake))
Action(Eat(Cake)) Pre: Have(Cake) Eff: not Have(Cake), Eaten(Cake)
Action(Bake(Cake)) Pre: not Have(Cake) Eff: Have(Cake)
```

Al livello S_1 , *Have(Cake)* (via persistenza) e *Eaten(Cake)* (via *Eat(Cake)*) sono **mutex** (l'unico modo di produrre l'uno è mutex con l'unico modo di produrre l'altro); in S_2 non lo sono più, perché compare anche *Bake(Cake)* come via alternativa per *Have(Cake)*, non mutex con la persistenza di *Eaten(Cake)*. **Extract-Solution** può quindi avere successo solo a partire da S_2 : in termini dello pseudocodice, **GP-Search** riceve $g = \{Have(Cake), Eaten(Cake)\}$ e $\pi_i = \emptyset$. Primo tentativo: sceglie da g il letterale *Eaten(Cake)*, il cui unico **resolver** in A_1 è $a = Eat(Cake)$; pone $\pi_i = \{Eat(Cake)\}$ e ricorre su $g = \{Have(Cake)\}$. Qui i possibili **resolvers** per *Have(Cake)* sono *Bake(Cake)* e la persistenza di *Have(Cake)*, ma *entrambi* risultano mutex con l'azione già scelta in π_i (*Eat(Cake)*): dopo il filtro **resolvers** = $\emptyset$, quindi **fallimento** – che si propaga fino alla scelta iniziale del resolver per *Eaten(Cake)*. Poiché anche

quella era l'unica opzione disponibile, il fallimento risale fino a **Extract-Solution**, che lo registra in **no-good** e forza a backtrackare sulla *scelta del letterale* da risolvere per primo. Secondo tentativo: si sceglie da g il letterale $Have(Cake)$ per primo, con resolver $a = \text{Bake}(Cake)$ (non mutex con nulla, $\pi_i = \{\text{Bake}(Cake)\}$); si ricorre poi su $g = \{\text{Eaten}(Cake)\}$, il cui resolver è ora la **persistenza** di $\text{Eaten}(Cake)$ (compatibile con $\text{Bake}(Cake)$, a differenza di quanto accadeva con $\text{Eat}(Cake)$): $\pi_i = \{\text{Bake}(Cake), \text{persistenza}\}$, $g = \emptyset$, e il regresso sulle precondizioni di π_i porta a un sottogoal risolvibile interamente in A_0 . Il piano estratto è

$$\pi = [\text{Eat}(Cake); \text{Bake}(Cake)],$$

che infatti realizza correttamente $\neg Have \wedge Eaten$ dopo il primo passo e $Have \wedge Eaten$ dopo il secondo.

14.5 Terminazione

Perché continuare a espandere il grafo anche dopo che si è livellato in termini di letterali? Perché, con esecuzione sequenziale (non parallela), l'applicazione di più azioni indipendenti a un livello può comunque richiedere più passi per essere serializzata (es. trasportare n pacchi uno alla volta con un solo aereo, dove il grafo si livella già a profondità 4 ma servono $4n-1$ espansioni per estrarre un piano fattibile). Il criterio corretto si basa su proprietà **monotone**: letterali e azioni crescono monotonamente (una volta comparsi, restano per sempre); mutex e no-good **decregono** monotonamente. Ne segue che deve esistere un livello in cui letterali e azioni coincidono con quelli del livello precedente, e un livello in cui mutex e no-good coincidono con quelli precedenti: quando *entrambe* le condizioni si verificano senza che una soluzione sia stata trovata, si può concludere con **fallimento** definitivo.

14.6 Proprietà

GraphPlan è corretto e completo: restituisce *failure* solo se il problema non è risolubile, altrimenti una soluzione come sequenza di insiemi di azioni, e termina sempre. GraphPlan è anche un **Partial-Order Planner**: le azioni allo stesso livello (non mutex per costruzione) possono eseguire in parallelo, e ogni linearizzazione compatibile con l'ordine dei livelli è una soluzione classica valida.

Molti pianificatori combinano un algoritmo di ricerca con una **funzione euristica** che stima la "bontà" di una scelta (in progression, la distanza dallo stato scelto al goal; in regression, la distanza dal sotto-goal scelto allo stato iniziale). Il principio generale è il **relaxing**: si definisce un problema *rilassato* (semplificato) analogo all'originale, lo si risolve (magari in modo approssimato), e si usa il costo di quella soluzione come euristica. Tecniche di rilassamento: **delete relaxation** (si cancellano gli effetti negativi delle azioni); **astrazione** (si riduce il problema, es. con meno oggetti); **landmark** (un insieme di azioni/fatti di cui ogni soluzione deve contenerne almeno uno, utile per un *pruning sicuro*).

15.1 Euristiche per la Ricerca in Avanti

Ignorando gli effetti negativi, la transizione rilassata è $\gamma^*(s, a) = s \cup effects^+(a)$. La distanza rilassata tra s e una proposizione p è $\Delta_0(s, p) = 0$ se $p \in s$, altrimenti $\Delta_0(s, p) = \min_a \{1 + \Delta_0(s, precondition(a)) \mid p \in effects^+(a)\}$ (o ∞ se nessuna azione produce p); per una congiunzione $g = p_1 \wedge \dots \wedge p_n$, $\Delta_0(s, g) = \sum_{p_i \in g} \Delta_0(s, p_i)$, da cui $h_0(s) = \Delta_0(s, g)$. Questa somma **non è ammissibile** (può contare più volte lo stesso costo condiviso tra sotto-obiettivi che interagiscono positivamente), ed è anche costosa da calcolare (un problema rilassato per ogni stato visitato). La stessa idea si applica simmetricamente alla **ricerca all'indietro** (regression): ignorando gli effetti negativi anche lì, si precompila una volta sola la distanza rilassata $\Delta_0(s_0, p)$ di *ogni* proposizione p dallo stato iniziale s_0 , e si usa per stimare, tra i possibili resolver di un sotto-goal, quello che minimizza $\Delta_0(s_0, precondition(a))$ – un vantaggio pratico notevole, perché s_0 è fisso per tutta la ricerca e quindi queste distanze vanno calcolate una sola volta, non ad ogni stato come in avanti.

Raffinamenti più accurati considerano il **massimo** anziché la somma: $\Delta_1(s, p) = \min_a \{1 + \max\{\Delta_1(s, q) \mid q \in precondition(a)\} \mid p \in effects^+(a)\}$, con $h_1(s, g) = \max\{\Delta_1(s, p) \mid p \in g\}$ – a parole, il costo per produrre p è dominato dalla sua condizione *più costosa* da soddisfare (più 1 per l'azione stessa), e tra tutte le azioni che potrebbero produrre p si sceglie la più economica secondo questo criterio; h_1 è ammissibile ma poco informativa, perché considera solo la distanza dal singolo letterale del goal più lontano, ignorando gli altri. h_2 raffina ulteriormente considerando la massima distanza di ogni **coppia** di proposizioni del goal, definita ricorsivamente in modo analogo a Δ_1 ma su coppie: $\Delta_2(s, \{p, q\}) = 0$ se $\{p, q\} \subseteq s$, altrimenti

$$\Delta_2(s, \{p, q\}) = \min_{a: \{p, q\} \subseteq effects^+(a)} \left(1 + \max\{\Delta_2(s, \{r, r'\}) \mid r, r' \in precondition(a)\} \right),$$

cioè – proprio come Δ_1 considerava la condizione singola più costosa – Δ_2 considera, tra le sole azioni che producono *entrambi* p e q insieme come effetto, quella la cui coppia di precondizioni è minima; $h_2(s, g) = \max\{\Delta_2(s, \{p, q\}) \mid p, q \in g\}$. Rispetto a h_1 (che valuta un letterale alla volta), h_2 cattura così le interazioni tra coppie di sotto-obiettivi che devono essere prodotte "in sinergia" dalla stessa azione: è ammissibile e più informativa di h_1 , ma più costosa da calcolare; l'idea si generalizza a un k qualsiasi, con costo crescente in k .

Un'euristica particolarmente efficace, usata da pianificatori come FF, è costruire un **planning graph rilassato** (senza effetti negativi né mutex): il costo h_G di un goal è la profondità del primo livello in cui tutti i suoi letterali compaiono insieme. È ammissibile, simile a h_2 ma più economica da calcolare e precompilare (dipende solo da dominio e stato iniziale, quindi riusabile per goal diversi con lo stesso s_0).

Un concetto trasversale, già anticipato a proposito dei landmark, è il **safe pruning** (potatura sicura): se si conosce già una soluzione di costo k (ad esempio la prima trovata da una ricerca

greedy), qualunque nodo u della frontiera con $h(u) > k$ può essere scartato senza rischio, perché nessun suo completamento potrà mai migliorare la soluzione già nota – a differenza di un pruning euristico generico, questo è "sicuro" nel senso che non può mai eliminare per errore l'unica soluzione ottima.

15.2 Euristiche per la Ricerca nello Spazio dei Piani

Il Least-Commitment Planning si può vedere come una **ricerca AND/OR**: i **flaw** sono rami AND (vanno risolti tutti), i possibili **resolver** per un flaw sono rami OR (ne basta uno). L'ordine di selezione dei flaw non cambia la forma finale della soluzione, ma cambia drasticamente la dimensione dell'albero esplorato: la strategia **FAF** (*Fewer Alternatives First*) preferisce sempre il flaw con **meno resolver**, perché riduce i costi di un eventuale backtracking e rende più facile scoprire presto un flaw irrisolvibile – con un vantaggio empirico marcato rispetto a scegliere prima i flaw con molti resolver. Per la scelta del resolver, un'euristica tipica stima per regressione (con al più k passi) la distanza del sotto-goal residuo dallo stato iniziale, preferendo il resolver che la minimizza.

I piani reali hanno spesso una **struttura regolare** (in un dominio logistico, un **load** è sempre seguito, prima o poi, da un **unload**): questa struttura si può catturare come **piani astratti** o **macro-azioni**, tipicamente indipendenti tra loro. Le **HTN** sfruttano questa indipendenza cercando in uno **spazio astratto** di task, anziché nello spazio degli stati. Esempio (pianificare un viaggio per una conferenza): il task "vai in aeroporto" si decompone in "se hai tempo, mezzi pubblici, altrimenti macchina"; "prendi l'aereo" si decompone in check-in, sicurezza, gate; "vai all'hotel" si decompone in "se hai soldi, taxi, altrimenti biglietto mezzi pubblici + fermata".

Gli elementi di un sistema HTN sono i **primitive operator** (plan operator alla STRIPS, direttamente eseguibili) e i **compound operator** o **metodi** (hanno precondizioni ed effetti, e codificano la decomposizione di un task in sotto-task); una soluzione è un piano completamente istanziato che contiene solo operatori primitivi. L'algoritmo, per intuizione, prende in input un **task astratto** (non un goal) e ripete: seleziona un metodo applicabile, usa il metodo per espandere il piano corrente scegliendo uno dei suoi sotto-piani – concettualmente simile al Least-Commitment Planning, dove un flaw è ora un sub-task non ancora risolto.

I **vantaggi**: i metodi codificano sia conoscenza di dominio sia conoscenza di problem-solving; HTN è teoricamente più espressivo del planning classico, ed è **Turing completo** (loop tramite invocazione ricorsiva dello stesso metodo, scelte if-then tramite decomposizioni alternative dello stesso metodo). Gli **svantaggi**: resta **NP-completo** nel caso peggiore, e richiede un maggiore sforzo di modellazione (non basta definire gli operatori, occorre anche definire come un metodo si decompone).

Un **sistema esperto** (o *knowledge-based system*) è "un sistema che emula la capacità decisionale di un esperto umano in un dominio ristretto" [Giarratano & Riley]; più precisamente, "un programma intelligente che usa conoscenza e procedure di inferenza per risolvere problemi difficili abbastanza da richiedere una significativa competenza umana". La potenza di un tale sistema, per Feigenbaum, dipende primariamente dalla **quantità e qualità della conoscenza** che possiede sul problema, non dalla sofisticazione dell'algoritmo: il programma non è una sequenza fissa di istruzioni, ma un **ambiente** in cui rappresentare, usare (*reasoning*) e modificare una base di conoscenza. Cinque proprietà lo caratterizzano: specificità del dominio, rappresentazione esplicita della conoscenza, meccanismi di ragionamento, capacità di spiegazione, capacità di operare in domini poco strutturati.

17.1 Architettura

Componente	Ruolo
Knowledge-Base (KB)	regole condizione-azione (<i>if-then</i> , o premessa-conseguenza)
Working Memory (WM)	fatti iniziali e quelli generati dalle inferenze
Inference Engine	pattern matching + agenda + execution (vedi sotto)
Explanation Facility	giustifica le soluzioni tramite la catena di regole attivate
Knowledge Acquisition	assiste l'integrazione/manutenzione di nuove regole
User Interface	pone problemi e restituisce risposte

Il motore inferenziale procede in tre fasi cicliche: **pattern matching** (confronta la parte *if* di ogni regola con i fatti della WM; le regole soddisfatte sono *attivabili* e finiscono nell'**agenda**); l'**agenda**, lista ordinata di regole attivabili (per priorità o altra strategia di preferenza); l'**execution**, che esegue (*fire*) la regola in cima all'agenda.

17.2 Forward vs. Backward Chaining

	Forward chaining	Backward chaining
Tipo	<i>data-driven</i>	<i>goal-driven</i>
Meccanismo	dai fatti alle conclusioni	da un'ipotesi, si cercano fatti/regole a supporto
Uso tipico	monitoraggio/controllo real-time	sistemi diagnostici
Esempi	CLIPS, OPS5	MYCIN, Prolog

17.3 Domini Applicativi

Compito	Descrizione	Esempio
Interpretazione	analisi di dati rumorosi per determinarne il significato	Dendral, Hearsay-II
Diagnosi	analisi di dati per identificare malattie/errori	Mycin
Monitoring	interpretazione continua, allarmi in tempo reale	VM
Planning & Scheduling	sequenza intelligente di azioni per un obiettivo	Molgen
Previsione	prevedere il futuro da un modello del passato	Prospector
Progetto/configurazione	progettare sistemi da specifiche	R1/XCON

Un sistema esperto **non** è indicato quando: esistono algoritmi tradizionali efficienti per il problema; l'aspetto centrale è la computazione, non la conoscenza; la conoscenza non è modellabile/elicitabile efficientemente; l'utente finale è comprensibilmente riluttante per la criticità del task (es. dosaggi medici, pilotaggio). Un sistema a regole di produzione ha la stessa potenza espressiva di una macchina di Turing (può quindi in linea di principio risolvere qualsiasi problema calcolabile), ma non è sempre il formalismo più adatto.

CLIPS ('C' Language Integrated Production System) è un motore forward-chaining sviluppato dalla NASA tra il 1985 e il 1996 (tuttora mantenuto), ideale quando un problema complesso può sfruttare conoscenza esplicita, anche euristica. È detto **eterarchico** (piatto: nessuna gerarchia di controllo imposta a priori). Il ciclo di esecuzione realizza esattamente il forward chaining descritto sopra:

1. determina tutte le regole **attivabili** (antecedente soddisfatto);
2. le ordina in **agenda**;
3. esegue (*fire*) la regola in cima, modificando i fatti in WM;
4. ripete finché non ci sono più regole attivabili.

Il risultato è ottenuto per **concatenazione di regole**, che è essa stessa una spiegazione della conclusione: i fatti generati da una regola diventano input per il pattern matching di regole successive.

18.1 Fatti

Un **fatto** è un'unità indivisibile di informazione, identificata da un nome di relazione con zero o più **slot**. I fatti **ordinati** non hanno slot dichiarati; quelli **non ordinati** si dichiarano con `deftemplate`:

```
; fatto ordinato
(person-name Franco L Verdi)

; deftemplate e fatto non ordinato
(deftemplate person "commento opzionale"
  (slot name)
  (slot age (type INTEGER) (default 18) (range 0 ?VARIABLE))
  (slot gender (type SYMBOL) (allowed-symbols male female)))

(person (name "Franco L. Verdi") (age 46)
  (eye-color brown) (hair-color brown))
```

(range 0 ?VARIABLE) indica nessun limite superiore; `allowed-symbols` restringe i valori ammessi. Un `multislot` accetta più valori, eventualmente con `cardinality` minima/massima: (multislot player (type STRING) (cardinality 6 6)).

Il costrutto `deffacts` definisce l'insieme di fatti veri all'avvio: sono effettivamente asseriti in WM solo dopo il comando (`reset`). Comandi principali: (`assert <fatto>+`) aggiunge fatti; (`retract <indice>+`) li rimuove; (`modify <indice> (slot valore)+`) equivale a un `retract` seguito da un nuovo `assert` (con indice incrementato); (`facts`) elenca i fatti in WM; (`watch facts`)/(`unwatch facts`) tracciano i cambiamenti.

18.2 Regole

```
(defrule <nome-regola> ["commento"]
  <pattern>* ; antecedente (LHS)
=>
  <azione>* ; conseguente (RHS)
```

Esempio con pattern completamente istanziato:

```
(defrule birthday-FLV
  (person (name "Luigi") (age 46) (eye-color brown) (hair-color brown))
  (date-today April-13-02)
=>
  (printout t "Happy birthday, Luigi!")
  (assert (there-s-a-party)))
```

Questa regola non ha variabili: il suo antecedente matcha solo ed esclusivamente un fatto **person** con *esattamente* quei valori (Luigi, 46 anni, occhi e capelli castani) insieme al fatto (**date-today April-13-02**); se in WM esistono entrambi, la regola si attiva, ed eseguendola stampa il messaggio e asserisce un nuovo fatto (**there-s-a-party**) (che a sua volta potrebbe attivare altre regole a valle). È un caso limite, utile solo per introdurre la sintassi: nella pratica una regola così rigida (nessuna variabile) è utile solo per un singolo fatto specifico, mentre l'utilità di CLIPS emerge con le variabili, che permettono a una regola di scattare per interi insiemi di fatti diversi.

In generale si usano **variabili** (simboli che iniziano con ?): ogni occorrenza della stessa variabile in una regola deve avere lo stesso valore (determinato dalla prima occorrenza, più a sinistra), e il binding è locale alla regola. Le **wildcard** ? (un singolo valore qualsiasi) e \$? (zero o più valori, in un multislot) non catturano un nome. Una variabile può anche fungere da "maniglia" su un intero fatto, per poterlo poi modificare:

```
(defrule birthday-FLV
  ?person <- (person (name "Luigi") (age 46) ...)
  (date-today April-13-02)
=>
  (printout t "Happy birthday, Luigi!")
  (modify ?person (age 47)))
```

Qui **?person** non contiene il *valore* di uno slot, ma è legata all'intero fatto **person** che ha soddisfatto il pattern (il suo indice/handle in WM): quando la regola scatta, (**modify ?person (age 47)**) ritira quel fatto e ne asserisce uno identico tranne che per lo slot **age**, ora a 47 – realizzando così il "compleanno" senza dover riscrivere l'intero fatto a mano. Da notare una conseguenza pratica della rifrazione: dopo il **modify**, il fatto (con nuovo indice) non matcha più il pattern originale (**age 46**), quindi la regola non può ri-scattare all'infinito sullo stesso Luigi.

Un meccanismo fondamentale è la **rifrazione**: impedisce che una regola scatti due volte sugli stessi identici fatti, anche se il pattern matcha più fatti (es. una regola che stampa "X ha gli occhi blu" scatta una volta per ciascuna persona con occhi blu, non all'infinito). Comandi utili: (**rules**), (**ppdefrule nome**), (**agenda**), (**watch rules**), (**watch activations**); per il debug, (**dribble-on file**)/(**dribble-off**) e (**set-break regola**)/(**remove-break regola**)/(**show-breaks**).

18.3 Un Esempio Completo: Caricamento ed Esecuzione

```
(defrule start
  (greet) => (printout t "hello" crlf))
(deffacts MyFacts (greet))
```

```
CLIPS> (load today.clp)
CLIPS> (facts)
f-0 (initial-fact)
CLIPS> (reset)
CLIPS> (facts)
f-0 (initial-fact)
f-1 (greet)
CLIPS> (run)
hello
```

(run) esegue il motore fino a quando l'agenda si svuota o una regola esegue (halt); (run n) limita l'esecuzione a n attivazioni.

18.4 Pattern Matching Avanzato

Oltre al matching diretto, i **field constraint** filtrano il valore di un singolo campo: & (and, es. ?name&Harvey), ~ (not, es. ~Simpson), | (or, es. Simpson|Oswald), : (vincolo generico via espressione, es. ?age&:(> ?age 20)). In alternativa, la clausola (test <espressione-booleana>) esprime lo stesso vincolo in modo indipendente dal pattern:

```
(defrule adult-blue-eyes
  (person (name ?name) (age ?age&:(>= ?age 18)))
  (eye-color ?eyes&blue|green)
=>
  (printout t ?name " e' maggiorenne con occhi chiari" crlf))
```

Per ogni fatto **person** in WM, il vincolo ?age&:(>= ?age 18) lega prima ?age al valore dello slot **age**, poi verifica la condizione (>= ?age 18): solo se vera il matching prosegue; il secondo pattern richiede inoltre un fatto **eye-color** con valore **blue** o **green**. Se entrambi i pattern trovano riscontro, la regola scatta stampando, per ogni ?name che soddisfa la congiunzione, la frase "?name e' maggiorenne con occhi chiari". Il field constraint e la clausola (test ...) equivalente ((person (name ?name) (age ?age)) (test (>= ?age 18))) sono semanticamente intercambiabili, ma il field constraint ha un vantaggio pratico: legando il vincolo direttamente al campo, il motore può usarlo per indicizzare/filtrare il pattern matching più efficacemente (scartando subito i fatti che non soddisfano il vincolo), mentre un test separato viene valutato solo dopo che tutti i pattern precedenti hanno già trovato un match.

Sui **multislot**: il pattern (data \$?multi) cattura l'intera coda; pattern come (data ?blue red \$?) o (data \$?prima blue \$?dopo) permettono di cercare un valore fisso in mezzo a un multislot di lunghezza variabile.

Funzioni built-in fondamentali sui multislot/multifield: **length\$** (numero di elementi), **member\$** (appartenenza) e **delete-member\$** (rimozione), **insert\$** (inserimento in posizione), **first\$/rest\$** (primo elemento / resto), **replace\$** (sostituzione di un intervallo), **nth\$** (elemento in posizione n), **delete\$** (rimozione di un intervallo), **explode\$** (multifield da stringa), **subsetp** (test sottoinsieme), **subseq\$** (estrazione di sottosequenza):

```
CLIPS> (first$ (create$ a b c))
(a)
CLIPS> (rest$ (create$ a b c))
(b c)
CLIPS> (replace$ (create$ drill wrench pliers) 3 3 machete)
(drill wrench machete)
CLIPS> (delete$ (create$ hammer drill saw pliers wrench) 3 4)
(hammer drill wrench)
CLIPS> (subsetp (create$ hammer saw) (create$ hammer drill saw pliers))
TRUE
```

18.5 Strategie di Conflict Resolution

L'**agenda** funziona come uno stack (la regola in cima è la prima eseguita); ogni modulo (vedi sotto) ha la propria. La posizione di una regola attiva è determinata, in ordine: (a) **salience** (priorità esplicita, intervallo $[-10000, 10000]$, dichiarata con (declare (salience N)) – non è buona norma usarla per forzare un ordine preciso); (b) a parità di salience, la **strategia di conflict resolution**; (c) a parità di tutto, l'ordine di scrittura delle regole. Le strategie disponibili ((set-strategy <nome>), (get-strategy)):

Strategia	Criterio
depth (default)	regole attivate da fatti più recenti in cima
breadth	regole attivate da fatti più recenti in fondo
simplicity	a parità di salience, meno specifiche prima
complexity	a parità di salience, più specifiche prima
lex	confronta i <i>time-tag</i> di tutti i fatti che attivano la regola
mea (<i>means-ends analysis</i>)	confronta prima il time-tag del fatto che soddisfa il <i>primo</i> pattern
random	valore casuale assegnato a ogni nuova attivazione

dove la **specificity** è determinata dal numero di confronti/binding nella LHS della regola.

Il meccanismo di **lex** e **mea** in dettaglio: ogni fatto in WM riceve un **time-tag** crescente (un contatore incrementato a ogni **assert/modify**, che indica quanto è "recente"); per ogni attivazione si costruisce la lista dei time-tag dei fatti che l'hanno generata, **ordinata dal più recente al meno recente**. Con **lex**, due attivazioni si confrontano scorrendo entrambe le liste ordinate in parallelo, posizione per posizione: vince (va in cima all'agenda) l'attivazione che ha il time-tag più alto alla prima posizione in cui le due liste differiscono. Con **mea** si confronta invece *solo* il time-tag del fatto che soddisfa il primo pattern della LHS – se questo basta a decidere si conclude subito, altrimenti si passa alla strategia successiva a parità. Esempio: siano attive due regole, **rule1** innescata dai fatti con time-tag ($f-3, f-1$) e **rule2** dai fatti ($f-2, f-5$); ordinando dal più recente: **rule1** → (3,1), **rule2** → (5,2). Con **lex**, si confronta la prima posizione: $5 > 3$, quindi **rule2** vince ed esegue per prima. Con **mea**, si guarda invece solo il time-tag del fatto che soddisfa il *primo pattern scritto nella regola* (non necessariamente il più recente della lista): se per **rule1** è $f-1$ e per **rule2** è $f-5$, allora $5 > 1$ e vince ancora **rule2** – ma con altri fatti l'esito di **lex** e **mea** può facilmente divergere, proprio perché guardano criteri diversi.

18.6 Espressioni Antecedenti Complesse

Per default i pattern nella LHS sono in congiunzione implicita (**and**); sono disponibili anche:

```
(defrule celebrate
  (or (birthday) (anniversary)) => (printout t "Festa!" crlf))

(defrule working-day
  (not (birthday)) (not (anniversary)) => (printout t "Si lavora." crlf))

(defrule emergency-report
  (exists (or (emergency (type fire)) (emergency (type bomb))))
=> (printout t "C'è un'emergenza." crlf))

(defrule evacuated-all
  (forall (emergency (type fire|bomb) (location ?b))
    (evacuated (building ?b)))
=> (printout t "Tutti gli edifici in emergenza sono evacuati." crlf))
```

or scatta se *almeno uno* dei pattern al suo interno matcha (nell'esempio, **celebrate** si attiva se in WM c'è un fatto **birthday** oppure un fatto **anniversary**, non necessariamente entrambi); **not** scatta se il pattern al suo interno *non* matcha alcun fatto (**working-day** si attiva solo quando né **birthday** né **anniversary** sono presenti in WM – è la negazione per fallimento, analoga concettualmente a $\neg$ in Prolog). **exists** scatta (una sola volta, per rifrazione) se *almeno un* fatto soddisfa il pattern; **forall** scatta solo se, per *ogni* fatto che soddisfa il primo pattern, vale anche il secondo (utile per verificare condizioni universali sulla WM). Ad esempio, con i fatti (**emergency (type fire) (location west-wing)**) e (**evacuated (building west-wing)**) in WM, **evacuated-all** scatta (l'unico edificio in emergenza è anche evacuato); se invece in WM ci fosse anche (**emergency (type bomb) (location east-wing)**) senza un corrispondente (**evacuated (building east-wing)**), la regola *non* scatterebbe, perché esiste un edificio in emergenza (**east-wing**) per cui il secondo pattern non trova riscontro.

18.7 Modellare un Mondo Dinamico

I fatti non possono essere annidati (uno slot non può contenere un altro fatto): per collegare informazioni correlate (es. una persona e il suo indirizzo, o più fatti che descrivono lo *stesso stato* in una ricerca) si usa un **identificatore comune**, spesso generato con (`gensym`) (simbolo `genN`, non garantito univoco) o (`gensym*`) (garantito univoco, controlla la WM):

```
(deftemplate block (slot state-id) (slot name) (slot on-top-of) (slot below-of))

(defrule grasp-on-top-of
  (current-state-id ?id)
  ?h <- (hand (state-id ?id) (holding NONE))
  ?b1 <- (block (state-id ?id) (name ?block) (on-top-of ?b1&~?block) (below-of NONE))
  ?b2 <- (block (state-id ?id) (name ?b1) (on-top-of ?x) (below-of ?block))
=>
  (retract ?h ?b1 ?b2)
  (bind ?new (gensym*))
  (assert (next-state-id ?new)
    (hand (state-id ?new) (holding ?block))
    (block (state-id ?new) (name ?block) (on-top-of GRASPED) (below-of NONE))
    (block (state-id ?new) (name ?b1) (on-top-of ?x) (below-of NONE))))
```

Questa regola dice però *solo cosa cambia*: senza altro, tutto ciò che non compare esplicitamente andrebbe perso nel nuovo stato – il classico **frame problem**. Si risolve con regole di **persistenza**, che ricopiano nel nuovo stato tutto ciò che non è stato toccato:

```
(defrule persistency-block (declare (salience 10))
  (next-state-id ?new) (current-state-id ?id)
  ?b <- (block (state-id ?id) (name ?block))
  (not (block (state-id ?new) (name ?block)))
=> (duplicate ?b (state-id ?new)))

(defrule persistency-end (declare (salience 5))
  ?n <- (next-state-id ?new) ?o <- (current-state-id ?id)
=> (retract ?n ?o) (assert (current-state-id ?new)))
```

Il comando (`duplicate ?fatto (slot valore)+`) crea una **copia** del fatto legato a `?fatto`, identica in tutti gli slot tranne quelli esplicitamente sovrascritti: (`duplicate ?b (state-id ?new)`) duplica quindi il blocco `?b` dello stato vecchio, cambiando solo il suo `state-id` al nuovo stato – è il modo naturale per "copiare tutto ciò che non cambia" senza riscrivere a mano ogni slot. La condizione (`(not (block (state-id ?new) (name ?block)))`) è la guardia che rende la regola *sicura da rieseguire*: senza di essa, `persistency-block` scatterebbe per ogni blocco vecchio a ogni ciclo, duplicando anche i blocchi che la regola di dominio (`grasp-on-top-of`) ha *già* asserito esplicitamente nel nuovo stato (con valori *diversi*, es. `on-top-of GRASPED`), sovrascrivendoli erroneamente con la versione vecchia. La guardia verifica quindi, per ogni blocco del vecchio stato, che *non esista già* un blocco con lo stesso nome nel nuovo stato: solo in quel caso lo copia inalterato; i blocchi già ridefiniti dalla regola di dominio vengono correttamente lasciati come sono. La *salience* più alta garantisce che tutti gli elementi persistenti siano copiati *prima* che il nuovo stato diventi quello corrente. Il comando (`bind ?var espressione`) lega esplicitamente un valore a una variabile, anche "nuova" (non presente nella LHS), nel conseguente di una regola.

18.8 Input/Output

```
(open "example.dat" my-file "r") ; "w" per scrivere
(bind ?input (read my-file)) ; legge un token
(bind $?input (readline my-file)) ; legge un'intera riga (multifield)
(close my-file)
```

18.9 Programmazione Modulare

In programmi con migliaia di regole, la **conoscenza di controllo** (che regola il comportamento) interlacciata con la **conoscenza di dominio** rende la manutenzione difficile. Un tipico problema a fasi (es. diagnosi: *detection* → *isolation* → *recovery* → di nuovo *detection*) potrebbe usare solo la **salience** per separare le fasi, ma questo **non garantisce** l'ordine corretto (se in WM manca il fatto della fase più prioritaria, regole di fasi successive scattano comunque per prime). Un miglioramento è usare **fatti di controllo** espliciti ((*phase detection*), ecc., con regole a bassa salience che avanzano la fase), ma anche questo approccio ha svantaggi strutturali: la WM resta comunque unica e non partizionata realmente, e tornare a una fase precedente ri-asserendo il suo fatto di controllo rischia di riattivare troppe regole (la rifrazione impedirebbe di rieseguire quelle già scattate in precedenza sugli stessi fatti).

La soluzione strutturale sono i **moduli**:

```
(defmodule DETECTION)
(defmodule ISOLATION)
(defmodule RECOVERY)
(defrule DETECTION::rule-1 ... => ...)
```

Ogni modulo ha la propria WM e la propria agenda. I costrutti `defrule`/`deffacts` sono privati; i `deftemplate` possono essere condivisi tra moduli con `export/import`:

```
(defmodule module-name (export deftemplate ?ALL))
(defmodule module-name (import in-module deftemplate ?ALL))
```

Le regole prese in considerazione sono solo quelle del modulo che ha il **focus** in cima a uno stack dedicato: (`focus modulo`) lo impila; un modulo viene tolto dallo stack (`pop`) quando non ci sono più regole eseguibili, o esplicitamente con (`pop-focus`)/(`return`). (`reset`) e (`clear`) riportano il focus su MAIN. L'**auto-focus** ((`declare (auto-focus TRUE)`)) permette a una regola di essere considerata anche quando il suo modulo non ha il focus – utile per regole che riconoscono violazioni di vincoli e devono scattare comunque. Sostituendo i fatti di controllo con moduli+focus:

```
(deffacts MAIN::control-information
  (phase-sequence DETECTION ISOLATION RECOVERY))
(defrule MAIN::change-phase
  ?list <- (phase-sequence ?next $?others)
  => (focus ?next) (retract ?list) (assert (phase-sequence $?others ?next)))
```

molte meno regole, e il round-robin tra fasi è gestito naturalmente dallo stack del focus.

18.10 Ricerca con Backtracking in CLIPS

Le strutture stesse di CLIPS si prestano a implementare una **ricerca in profondità con backtracking**: uno **stato** è identificato da un id condiviso (come sopra); il **cammino aperto** è rappresentato dalla WM (fatti di più stati coesistono); le **alternative non ancora esplorate** sono rappresentate dall'**agenda** stessa. Con la strategia **depth**, le attivazioni sono stratificate per stato: far scattare una regola genera un nuovo stato successore, mentre le opzioni aperte restano in agenda; quando non ci sono più regole applicabili per lo stato corrente, l'esecuzione passa automaticamente alla prossima regola in cima allo stack – che riguarda uno stato antenato: un **backtracking implicito**, senza bisogno di codificarlo esplicitamente.

Esempio: assegnazione di *N* ospiti a sedie, vincolando sesso alternato e almeno un hobby condiviso coi vicini. Con un buon ordinamento delle scelte l'assegnazione può procedere senza mai bloccarsi; in generale, però, si arriva spesso a un vicolo cieco (nessuna regola applicabile per il prossimo posto): a quel punto "non essendoci altre regole applicabili da questo cammino, CLIPS riconsidera altri cammini aperti rimasti in WM" – il motore riprende automaticamente

da un'alternativa lasciata in agenda a un passo precedente, eventualmente ripetendo il processo più volte prima di trovare un'assegnazione completa valida.

In termini di strutture CLIPS, il problema si modella con un fatto per ogni ospite ancora da sistemare e un fatto **seated** per ogni sedia già assegnata, tutti taggati con lo stesso **state-id** visto nell'esempio dei blocchi:

```
(deftemplate guest (slot state-id) (slot name) (slot sex) (multislot hobbies))
(deftemplate seated (slot state-id) (slot chair) (slot name))

(defrule seat-guest
  (current-state-id ?id)
  ?g <- (guest (state-id ?id) (name ?n) (sex ?s) (hobbies $?h))
  ?prev <- (seated (state-id ?id) (chair ?c) (name ?pn))
  (guest (state-id ?id) (name ?pn) (sex ?ps&~?s) (hobbies $?ph))
  (test (> (length$ (intersect$ ?h ?ph)) 0)) ; almeno un hobby in comune
=>
  (bind ?new (gensym*))
  (assert (next-state-id ?new)
    (seated (state-id ?new) (chair (+ ?c 1)) (name ?n))))
```

Ogni istanziazione di **seat-guest** compatibile con l'ultima sedia occupata (**?prev**) genera un *nuovo* stato successore, esattamente come **grasp-on-top-of** nell'esempio dei blocchi; più ospiti compatibili in un dato momento significano più attivazioni in agenda, cioè più cammini alternativi non ancora esplorati. Se a un certo punto nessun ospite rimasto soddisfa il vincolo di sesso alternato e hobby condiviso rispetto all'ultima sedia occupata dello stato corrente, la regola **seat-guest** non ha più istanziazioni per quello stato: l'agenda passa automaticamente all'attivazione successiva rimasta, che corrisponde a uno stato-antenato con una scelta di ospite diversa – il backtracking, di nuovo, senza alcuna riga di codice dedicata a "tornare indietro".

18.11 Incertezza nei Sistemi a Regole: i Certainty Factor

CLIPS non ha un meccanismo nativo per il ragionamento incerto; si può però replicare l'approccio storico di **MYCIN**. Un **Certainty Factor** (Carnap, 1950) è definito come differenza tra una misura di crescita del *belief* e una di *disbelief*:

$$CF(H, E) = MB(H, E) - MD(H, E)$$

con valore in $[-1, +1]$: $+1$ = vero per certo, 0 = incertezza massima, -1 = falso per certo. A differenza della probabilità, un CF non presuppone che l'evidenza a favore di H dica nulla sul complementare $\neg H$: i medici, tipicamente, ragionano per evidenze qualitative a favore di un'ipotesi, non per distribuzioni di probabilità complete.

Per combinare i CF: nell'**antecedente**, $CF(P_1 \text{ and } P_2) = \min\{CF(P_1), CF(P_2)\}$, $CF(P_1 \text{ or } P_2) = \max\{CF(P_1), CF(P_2)\}$, $CF(\text{not } P) = -CF(P)$ (con la regola pratica che, se il CF risultante è < 0.2 , la regola è considerata inapplicabile); il **CF del fatto prodotto** è il CF dell'antecedente moltiplicato per il CF "a priori" della regola. Quando la stessa conclusione è raggiunta da più cammini di ragionamento con CF diversi CF_1, CF_2 , si fondono con

$$NewCF = \begin{cases} (CF_1 + CF_2) - (CF_1 \cdot CF_2) & \text{se entrambi } \geq 0 \\ (CF_1 + CF_2) + (CF_1 \cdot CF_2) & \text{se entrambi } < 0 \\ \frac{CF_1 + CF_2}{1 - \min\{|CF_1|, |CF_2|\}} & \text{se di segno opposto} \end{cases}$$

(verifica: $CF_1=0.7, CF_2=0.5 \Rightarrow (0.7+0.5) - (0.7 \times 0.5) = 1.2 - 0.35 = 0.85$). In CLIPS si rappresentano i fatti nel formato **object-attribute-value** (OAV) con un campo CF:

```

(deftemplate OAV::oav (multislot object) (multislot attribute) (multislot value)
  (slot CF (type FLOAT) (range -1.0 +1.0)))

(defrule OAV::combine-certainties-both-positive
  (declare (auto-focus TRUE))
  ?f1 <- (oav (object $?o) (attribute $?a) (value $?v) (CF ?C1&:(>= ?C1 0)))
  ?f2 <- (oav (object $?o) (attribute $?a) (value $?v) (CF ?C2&:(>= ?C2 0)))
  (test (neq ?f1 ?f2))
=>
  (retract ?f1)
  (bind ?C3 (- (+ ?C1 ?C2) (* ?C1 ?C2)))
  (modify ?f2 (CF ?C3)))

```

`auto-focus TRUE` garantisce che due OAV identici vengano fusi prima di essere usati da altre regole (richiede anche `(set-fact-duplication TRUE)`, perché CLIPS per default non permette fatti duplicati).

18.12 Caso di Studio: Sistema Esperto per la Scelta del Vino

Un caso di studio completo (architettura a moduli `QUESTIONS`, `RULES`, `CHOOSE-QUALITIES`, `WINES`, `PRINT-RESULTS` – rispettivamente: porre le domande, applicare le regole di dominio, scegliere gli attributi con CF più alto, proporre i vini compatibili, stampare il risultato) mostra i CF applicati in scala $[0, 100]$ anziché $[-1, +1]$ (formule equivalenti a meno di riscalatura). Le regole di dominio sono **fatti**, non clausole CLIPS dirette, per poterle interpretare uniformemente:

```

(deftemplate RULES::rule (slot certainty (default 100.0)) (multislot if) (multislot then))

(rule (if tastiness is average)
  (then best-body is light with certainty 30 and
    best-body is medium with certainty 60 and
    best-body is full with certainty 30))

```

Le domande da porre all'utente sono anch'esse fatti (`QUESTIONS::question`, con attributo, testo, risposte valide, ed eventuali **precursors** che ne condizionano l'attivazione); una regola generica le pone tramite una funzione di validazione dell'input:

```

(defrule QUESTIONS::ask-a-question
  ?f <- (question (already-asked FALSE) (precursors) (the-question ?q)
    (attribute ?attr) (valid-answers $?va))
=>
  (modify ?f (already-asked TRUE))
  (assert (attribute (name ?attr) (value (ask-question ?q ?va)))))

```

Il modulo `RULES` "consuma" progressivamente l'antecedente di ogni regola man mano che le condizioni sono soddisfatte dagli attributi noti, accumulando il CF con `min` (congiunzione), e quando l'antecedente è vuoto asserisce la prima conseguenza (con CF proporzionale al prodotto tra CF accumulato e CF della singola conseguenza, diviso 100):

```

(defrule RULES::remove-is-condition-when-satisfied
  ?f <- (rule (certainty ?c1) (if ?attr is ?val $?rest))
  (attribute (name ?attr) (value ?val) (certainty ?c2))
=> (modify ?f (certainty (min ?c1 ?c2)) (if ?rest)))

(defrule RULES::perform-rule-consequent-with-certainty
  ?f <- (rule (certainty ?c1) (if) (then ?attr is ?val with certainty ?c2 $?rest))
=>
  (modify ?f (then ?rest))

```

```
(assert (attribute (name ?attr) (value ?val) (certainty (/ (* ?c1 ?c2) 100)))))
```

Infine, il modulo `WINES` propone i vini compatibili con gli attributi dedotti (colore, corpo, dolcezza preferiti), con CF pari al minimo dei CF dei singoli attributi – lo stesso principio di combinazione visto per l'antecedente delle regole:

```
(defrule WINES::generate-wines
  (wine (name ?name) (color $? ?c $?) (body $? ?b $?) (sweetness $? ?s $?))
  (attribute (name best-color) (value ?c) (certainty ?c1))
  (attribute (name best-body) (value ?b) (certainty ?c2))
  (attribute (name best-sweetness) (value ?s) (certainty ?c3))
  => (assert (attribute (name wine) (value ?name) (certainty (min ?c1 ?c2 ?c3)))))
```

Resta un ultimo caso da gestire: nel modulo `RULES`, più regole di dominio *distinte* possono dedurre lo stesso attributo con lo stesso valore (es. due indizi diversi che portano entrambi a "best-body is full"), ciascuna asserendo un proprio fatto `attribute` col proprio CF. Questi vanno fusi con la stessa formula di combinazione vista sopra (in scala 0–100 anziché $[-1, +1]$), tramite una regola generica analoga a `OAV::combine-certainties-both-positive` ma applicata ai fatti `attribute`:

```
(defrule MAIN::combine-certainties
  (declare (auto-focus TRUE))
  ?f1 <- (attribute (name ?n) (value ?v) (certainty ?c1))
  ?f2 <- (attribute (name ?n) (value ?v) (certainty ?c2))
  (test (neq ?f1 ?f2))
  =>
    (retract ?f1)
    (modify ?f2 (certainty (- (+ ?c1 ?c2) (/ (* ?c1 ?c2) 100)))))
```

La formula è concettualmente identica a quella del caso "entrambi positivi" già vista per l'OAV, riscalata per lavorare in $[0, 100]$ invece che in $[-1, +1]$ (da cui il termine $(/ (* ?c1 ?c2) 100)$ al posto di $(* ?c1 ?c2)$): senza questa fusione, il sistema tratterebbe erroneamente due conferme indipendenti dello stesso fatto come se fossero informazioni scorrelate da usare separatamente, anziché rafforzare la fiducia in un'unica conclusione.

19.1 Perché la Logica Classica non Basta

Un piano A_t per arrivare in aeroporto t minuti prima del decollo è minato da osservabilità parziale (traffico, meteo), osservazioni imprecise, incertezza sul risultato delle azioni (l'auto potrebbe non partire), e dalla difficoltà di modellare l'intero sistema. Un approccio puramente logico ha due esiti negativi: asserire con certezza " A_{25} funziona" porta a decisioni sbagliate; oppure si arriva a conclusioni troppo deboli per essere utili (" A_{25} funziona, se il ponte non è bloccato, se non piove, se..."), e persino A_{1440} (aspettare un giorno intero) non darebbe garanzie assolute.

Il problema non è solo pratico ma strutturale: nel dominio medico, la regola " $\forall p$, il sintomo mal di denti implica la malattia carie" è falsa (non ogni mal di denti è carie); estenderla con una disgiunzione di tutte le possibili cause è impraticabile (lista lunga, forse incompleta); e nemmeno la direzione opposta (" $\forall p$, la carie implica il sintomo mal di denti") è vera in generale. La relazione tra sintomo e malattia è *causale*, non di implicazione logica certa. I limiti della logica del primo ordine hanno tre cause: **pigrizia** (enumerare tutte le premesse/conseguenze è troppo oneroso), **ignoranza teorica** (non si conosce tutto il dominio), **ignoranza pratica** (anche conoscendo la teoria, possono mancare dati sul caso specifico).

19.2 La Probabilità come Soluzione

La probabilità riassume l'incertezza dovuta a pigrizia e ignoranza: assegnare una probabilità a un enunciato esprime un **grado di credenza** (*belief*) nella sua verità – non un grado di verità come nella logica fuzzy, e sempre relativo alle prove disponibili (**probabilità a priori** in assenza di prove, **a posteriori/condizionata** dopo osservazioni). Le decisioni razionali richiedono anche una nozione di utilità: **Teoria delle decisioni** = **Teoria delle Probabilità** + **Teoria dell'Utilità**, e un agente razionale massimizza l'utilità attesa (un'opzione con probabilità di successo maggiore non è automaticamente la scelta migliore).

19.3 Notazione e Assiomi

Una variabile casuale rappresenta una parte del mondo inizialmente sconosciuta, con un dominio (*range*) discreto o continuo; per convenzione le minuscole denotano variabili/valori generici, le maiuscole variabili specifiche, con l'abbreviazione booleana $cavity \equiv (Cavity=true)$. Gli **assiomi di Kolmogorov**: per ogni proposizione a, b ,

$$0 \leq P(a) \leq 1, \quad P(true) = 1, \quad P(false) = 0, \quad P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$

La loro giustificazione (argomento di *De Finetti*, o *Dutch book*): un agente i cui gradi di credenza violano questi assiomi è soggetto a una strategia di scommesse che gli fa perdere sistematicamente, qualunque cosa accada.

Un **evento atomico** è un assegnamento di valori a *tutte* le variabili casuali del dominio; gli eventi atomici sono mutuamente esclusivi ed esaustivi, e ogni proposizione è equivalente alla disgiunzione di tutti gli eventi atomici che la implicano, da cui

$$P(a) = \sum_{e_i \in \mathbf{e}(a)} P(e_i).$$

La **distribuzione di probabilità a priori** di una variabile è il vettore di probabilità per ciascun valore del suo dominio (normalizzato a somma 1); la **distribuzione congiunta** assegna una probabilità a ogni combinazione di valori di più variabili; la **congiunta completa** è costruita su tutte le variabili del dominio.

19.4 Probabilità Condizionata

$$P(a|b) = \frac{P(a \wedge b)}{P(b)} \quad (P(b) \neq 0)$$

da cui la **regola del prodotto**:

$$P(a \wedge b) = P(a|b)P(b) = P(b|a)P(a).$$

Attenzione: $P(a|b)$ non è un'implicazione logica probabilizzata – non coincide con $P(b \rightarrow a)$ (che ha senso solo in assenza di informazione su b), e non è monotona come l'implicazione (aggiungendo ulteriore informazione c , $P(a|b, c)$ può differire anche molto da $P(a|b)$, in entrambe le direzioni).

Esempio numerico (dentista: *toothache* = mal di denti, *catch* = lo strumento "prende", *Cavity* = carie), a partire dalla congiunta completa:

	toothache		¬toothache	
	catch	¬catch	catch	¬catch
cavity	0.108	0.012	0.072	0.008
¬cavity	0.016	0.064	0.144	0.576

si ottiene $P(\text{cavity} \vee \text{toothache}) = 0.28$; $P(\text{cavity}) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2$ (la **probabilità marginale**, ottenuta sommando su tutte le altre variabili – **regola di marginalizzazione** generale $\mathbf{P}(Y) = \sum_z \mathbf{P}(Y, z)$); e per condizionamento,

$$P(\neg \text{cavity} | \text{toothache}) = \frac{0.016 + 0.064}{0.108 + 0.012 + 0.016 + 0.064} = \frac{0.08}{0.2} = 0.4.$$

Questo tipo di calcolo, detto **inferenza per enumerazione**, si generalizza a $\mathbf{P}(X|\mathbf{e}) = \alpha \mathbf{P}(X, \mathbf{e}) = \alpha \sum_{\mathbf{y}} \mathbf{P}(X, \mathbf{e}, \mathbf{y})$ (con α costante di **normalizzazione** che riporta la somma a 1, E variabili di evidenza, Y variabili nascoste), ma ha due svantaggi: è costoso ($O(2^n)$ per n variabili booleane), e richiede comunque, in linea di principio, l'intera distribuzione congiunta.

19.5 Regola di Bayes

Dalla regola del prodotto:

$$P(b|a) = \frac{P(a|b) P(b)}{P(a)}$$

Perché è utile, se richiede comunque tre probabilità? Perché nei casi reali si dispone spesso dell'informazione nella forma "naturale" $P(\text{effetto}|\text{causa})$ (es. probabilità del sintomo data la malattia, nota dall'epidemiologia), mentre si vuole calcolare $P(\text{causa}|\text{effetto})$ (la diagnosi): anche se un sintomo è molto comune in una malattia rara, la probabilità che un paziente con quel sintomo abbia proprio quella malattia resta bassa, un effetto che Bayes cattura correttamente.

19.6 Indipendenza e Indipendenza Condizionale

a e b sono **indipendenti** se $P(a|b)=P(a)$ (equivalentemente $P(b|a)=P(b)$, o $P(a \wedge b)=P(a)P(b)$): l'indipendenza assoluta riduce drasticamente la dimensione della rappresentazione (es. n lanci di moneta indipendenti: $\mathbf{P}(C_1, \dots, C_n) = \prod_i \mathbf{P}(C_i)$, da $O(2^n)$ a $O(n)$), ma è rara nei domini reali. Più utile è l'**indipendenza condizionale**: *toothache* e *catch* non sono indipendenti (entrambi collegati a *Cavity*), ma lo sono *data Cavity* – se lo strumento "prende" è probabile che ci sia carie e quindi che il dente faccia male, ma la relazione passa sempre per la carie:

$$P(X, Y|Z) = P(X|Z) P(Y|Z)$$

Questo riduce

$$P(\text{Cavity}|\text{toothache}, \text{catch}) = \alpha P(\text{toothache}, \text{catch}|\text{Cavity}) P(\text{Cavity})$$

a

$$\alpha P(\text{toothache}|\text{Cavity}) P(\text{catch}|\text{Cavity}) P(\text{Cavity}):$$

da 7 probabilità indipendenti a 3. Generalizzando a una *Causa* con n *Effetti* condizionalmente indipendenti dati la causa si ottiene il **modello di Bayes ingenuo** (*naive Bayes*):

$$P(\text{Causa}, \text{Effetto}_1, \dots, \text{Effetto}_n) = P(\text{Causa}) \prod_i P(\text{Effetto}_i|\text{Causa}),$$

con rappresentazione $O(n)$ anziché $O(2^n)$ – "ingenuo" perché assume gli effetti indipendenti tra loro, anche quando nella realtà non sempre lo sono.

19.7 Reti Bayesiane: Sintassi e Semantica

Una **rete bayesiana** (*belief network*) è una rappresentazione grafica delle asserzioni di indipendenza condizionale di un dominio, ed è al tempo stesso una specifica concisa della sua intera distribuzione congiunta. È un **DAG** (grafo diretto aciclico): un nodo per variabile casuale; un arco $X \rightarrow Y$ indica che X ha un'influenza diretta su Y (X è *genitore* di Y); ogni nodo X_i porta una **tabella di probabilità condizionata** (CPT) $P(X_i | \text{Parents}(X_i))$ – per una variabile booleana con k genitori booleani, la CPT ha 2^k righe.

Esempio guida (allarme antifurto): John e Mary, vicini, chiamano se sentono l'allarme di casa; l'allarme può scattare per un'intrusione o per un piccolo terremoto (frequentissimi nella zona). Variabili: *Burglary*, *Earthquake* $\rightarrow$ *Alarm* $\rightarrow$ *JohnCalls*, *MaryCalls* (i vicini non percepiscono direttamente intrusioni/terremoti, solo l'allarme).

		B	E	$P(A B, E)$	A	$P(J A)$	A	$P(M A)$
$P(B)$	0.001	T	T	.95	T	.90	T	.70
$P(E)$	0.002	T	F	.94	F	.05	F	.01
		F	T	.29				
		F	F	.001				

19.8 Semantica Globale e Compattezza

La **semantica globale** definisce la congiunta completa come prodotto delle CPT locali:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

Per l'esempio dell'allarme, $P(j, m, a, \neg b, \neg e) = P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e) = 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \approx 0.00063$. La rete richiede solo $1+1+4+2+2 = 10$ valori, contro i $2^5 - 1 = 31$ della congiunta completa a 5 variabili booleane: in generale, se ogni variabile ha al più k genitori, la rete richiede $O(n \cdot 2^k)$ valori invece di $O(2^n)$ – una **compattezza** che cresce solo linearmente in n quando $k \ll n$ (tipico nei domini reali).

19.9 Semantica Locale e Markov Blanket

Equivalentemente (**semantica locale**), ogni nodo è condizionalmente indipendente dai suoi **non-discendenti** dati i suoi genitori. Più in generale, ogni nodo è condizionalmente indipendente da *tutti* gli altri nodi della rete data la sua **coperta di Markov** (*Markov blanket*): genitori, figli, e genitori dei figli.

19.10 Costruire una Rete Bayesiana

Per costruzione corretta si sceglie un ordinamento $X_n, \dots, X_1$ e, per ciascun X_i (dal più "vecchio" al più "nuovo"), si selezionano come genitori un sottoinsieme di $\{X_{i+1}, \dots, X_n\}$ tale che $\mathbf{P}(X_i | \text{Parents}(X_i)) = \mathbf{P}(X_i | X_{i+1}, \dots, X_n)$: questo garantisce la semantica globale, perché la **chain rule** $\mathbf{P}(X_1, \dots, X_n) = \prod_i \mathbf{P}(X_i | X_{i+1}, \dots, X_n)$ coincide allora esattamente con il prodotto delle CPT.

L'**ordine scelto conta**: costruendo l'esempio dell'allarme nell'ordine *non causale* M, J, A, B, E (chiedendosi ad ogni passo se la nuova variabile è indipendente, o condizionalmente indipendente, dalle precedenti) si ottiene una rete valida ma **più densa e meno intuitiva** ($M \rightarrow J$; $M, J \rightarrow A$; $A \rightarrow B$; $A, B \rightarrow E$) rispetto a quella causale ($B, E \rightarrow A \rightarrow J, M$): entrambe rispettano la semantica globale per costruzione, ma la rete non causale nasconde relazioni che, nella rete causale, sono immediate (es. B ed E indipendenti a priori), e richiede di specificare probabilità concettualmente più difficili da stimare direttamente (es. $P(E|B, A)$, quando causalmente B ed E sono cause indipendenti che convergono solo su A). **Regola pratica**: costruire la rete nell'ordine causale produce quasi sempre la rappresentazione più compatta e intuitiva.

19.11 Inferenza per Enumerazione

Per calcolare $P(B|j, m)$ (evidenza j, m ; variabili nascoste A, E):

$$P(B|j, m) = \alpha \sum_e \sum_a P(B)P(e)P(a|B, e)P(j|a)P(m|a) = \alpha P(B) \sum_e P(e) \sum_a P(a|B, e)P(j|a)P(m|a),$$

portando fuori dalle sommatorie i fattori che non dipendono dalla variabile sommata. Il numero di fattori nel prodotto cresce solo linearmente in n (grazie all'indipendenza condizionale), ma il numero di **addendi** nelle sommatorie resta esponenziale nel caso peggiore: **complessità** $O(n \cdot 2^n)$, con inoltre lo svantaggio pratico che **molti sotto-calcoli vengono ripetuti** più volte lungo rami diversi dell'espressione.

19.12 Eliminazione delle Variabili

L'algoritmo di **eliminazione delle variabili** applica la programmazione dinamica per evitare quella ripetizione: si esplicitano i **fattori** della formula (matrici indicizzate dalle variabili "libere", cioè non fissate dall'evidenza), e si valuta da destra a sinistra alternando due operazioni.

Le due operazioni fondamentali, con un piccolo esempio numerico di riscaldamento (non legato all'esempio dell'allarme, solo per fissare il meccanismo): il **prodotto puntuale** tra due fattori $f(X_1, \dots, X_j, Y_1, \dots, Y_k)$ e $g(Y_1, \dots, Y_k, Z_1, \dots, Z_l)$ produce $h = f \times g$ indicizzato dall'unione delle variabili, moltiplicando le celle che condividono gli stessi valori delle variabili in comune – se tutte booleane, h ha 2^{j+k+l} elementi: la dimensione di un fattore è **esponenziale** nel numero delle sue variabili, la vera fonte di complessità dell'algoritmo. Con $f(X, Y)$: $f(T, T)=.3$, $f(T, F)=.7$, $f(F, T)=.9$, $f(F, F)=.1$; e $g(Y, Z)$: $g(T, T)=.2$, $g(T, F)=.8$, $g(F, T)=.6$, $g(F, F)=.4$; il prodotto $h(X, Y, Z) = f(X, Y) \cdot g(Y, Z)$ vale, ad esempio, $h(T, T, T) = f(T, T) \cdot g(T, T) = .3 \times .2 = .06$ e $h(T, F, T) = f(T, F) \cdot g(F, T) = .7 \times .6 = .42$ (si noti che Y compare in entrambi i fattori originali e va quindi tenuta allo *stesso* valore in entrambi i fattori quando si moltiplica). Il **summing out** elimina poi una variabile sommando le sotto-matrici ottenute fissandola a ciascun valore: $\sum_x h(X, Y, Z) = h(x, Y, Z) + h(\neg x, Y, Z)$; ad esempio $h_2(T, T) = \sum_x h(x, T, T) = h(T, T, T) + h(F, T, T) = .06 + .18 = .24$.

Nell'esempio $P(B|j, m) = \alpha f_1(B) \times \sum_e f_2(E) \times \sum_a f_3(A, B, E) \times f_4(A) \times f_5(A)$, i fattori numerici (dalle CPT già date) sono $f_1(B) = (P(B)=.001, P(\neg B)=.999)$, $f_2(E) = (P(E)=.002, P(\neg E)=.998)$, $f_4(A) = (P(j|a), P(j|\neg a)) = (.90, .05)$, $f_5(A) = (P(m|a), P(m|\neg a)) = (.70, .01)$, e $f_3(A, B, E) = P(A|B, E)$ è la tabella $P(A|B, E)$ già vista (8 valori). Si elimina prima A : applicando prodotto puntuale e summing out come sopra, $f_6(B, E) = \sum_a f_3(a, B, E) f_4(a) f_5(a) = P(a|B, E) \times .63 + P(\neg a|B, E) \times .0005$ (poiché $f_4(a)f_5(a) = .9 \times .7 = .63$ e $f_4(\neg a)f_5(\neg a) = .05 \times .01 = .0005$);

numericamente, per le quattro combinazioni di B, E : $f_6(T, T) = .95 \times .63 + .05 \times .0005 \approx .5985$, $f_6(T, F) = .94 \times .63 + .06 \times .0005 \approx .5922$, $f_6(F, T) = .29 \times .63 + .71 \times .0005 \approx .1831$, $f_6(F, F) = .001 \times .63 + .999 \times .0005 \approx .0011$. Poi si elimina E da $f_2 \times f_6$: $f_7(B) = \sum_e f_2(e) f_6(B, e)$, cioè $f_7(T) = .002 \times .5985 + .998 \times .5922 \approx .5922$ e $f_7(F) = .002 \times .1831 + .998 \times .0011 \approx .0015$. Infine $P(B|j, m) = \alpha f_1(B) \times f_7(B)$: valori non normalizzati $\alpha^{-1} P(T|j, m) = .001 \times .5922 \approx .00059$ e $\alpha^{-1} P(F|j, m) = .999 \times .0015 \approx .00149$; normalizzando (dividendo per la somma $\approx .00208$) si ottiene $P(B|j, m) \approx \langle 0.284, 0.716 \rangle$ – coerente con l'intuizione che, avendo osservato entrambe le chiamate, un furto è comunque un evento raro rispetto a un semplice falso allarme (es. da terremoto), ma molto più probabile che a priori ($P(B)=0.001$).

Due ottimizzazioni pratiche: (1) **safe pruning** – ogni fattore che non dipende dalla variabile da eliminare può uscire dalla sommatoria senza essere moltiplicato, e ogni nodo **foglia** che non è né variabile di query né di evidenza può essere ignorato del tutto (es. calcolando $P(J|b)$, M è un nodo foglia irrilevante: $\sum_m P(m|a) = 1$ per definizione, quindi non contribuisce); regola generale (di nuovo dalla semantica locale): *ogni variabile che non è un progenitore di una variabile di query o di evidenza è irrilevante per l'interrogazione*; il procedimento è inoltre **ricorsivo**, perché rimuovere una foglia irrilevante può a sua volta rendere "foglia" (cioè priva di discendenti rimasti nel grafo) un'altra variabile in precedenza non eliminabile, permettendo di scartare anche quella. (2) l'**ordine di eliminazione** non cambia il risultato finale ma cambia enormemente la dimensione dei fattori intermedi generati; trovare l'ordine ottimo è intrattabile in generale, e si ricorre a un'euristica greedy (elimina per prima la variabile che minimizza la dimensione del prossimo fattore).

19.13 MPE e MAP

Due ulteriori task di inferenza, oltre al semplice calcolo di $P(X|\mathbf{e})$: la **Most Probable Explanation** (MPE), l'istanza più probabile di *tutte* le variabili del dominio data l'evidenza,

$$MPE(\mathbf{e}) = \arg \max_{\mathbf{x}} P(\mathbf{x}, \mathbf{e}),$$

e il **Maximum a Posteriori** (MAP), l'istanza più probabile di un sottoinsieme M di variabili di query,

$$MAP(\mathbf{e}) = \arg \max_{\mathbf{m}} P(\mathbf{m}, \mathbf{e}).$$